A Brief History of C

The C programming language was developed at Bell Labs during the early 1970's. Quite unpredictably it derived from a computer language named B and from an earlier language BCPL. Initially designed as a system programming language under UNIX it expanded to have wide usage on many different systems. The earlier versions of C became know as K&R C after the authors of an earlier book, "The C Programming Language" by Kernighan and Ritchie. As the language further developed and standardized, a version known as ANSI (American National Standards Institute) C became dominant. As you study this language expect to see references to both K&R and ANSI C. Although it is no longer the language of choice for most new development, it still is used for some system and network programming as well as for embedded systems. More importantly, there is still a tremendous amount of legacy software still coded in this language and this software is still actively maintained.

A Brief History of C++

Bjarne Stroustrup at Bell Labs initially developed C++ during the early 1980's. It was designed to support the features of C such as efficiency and low-level support for system level coding. Added to this were features such as classes with inheritance and virtual functions, derived from the Simula67 language, and operator overloading, derived from Algol68. Don't worry about understanding all the terms just yet, they are explained in About's C++ tutorials. C++ is best described as a superset of C, with full support for object-oriented programming. This language is in wide spread use.

Differences between C and C++

Although the languages share common syntax they are very different in nature. C is a procedural language. When approaching a programming challenge the general method of solution is to break the task into successively smaller subtasks. This is known as top-down design. C++ is an object-oriented language. To solve a problem with C++ the first step is to design classes that are abstractions of physical objects. These classes contain both the state of the object, its members, and the capabilities of the object, its methods. After the classes are designed, a program is written that uses these classes to solve the task at hand.

C is a compiled language. The C compiler is a program that reads source code, which is the C code written by a programmer, and produces an executable or binary file that in a format that can be read and executed (run) by a computer. The source file is a plain text file containing your code. The executable file consists of machine code, 1's and 0's that are not meant to be understood or read by people, but only by computers.

The best way to learn anything is to jump right in, so let's start by writing a simple C program.

First Program - Hello World

```
#include <stdio.h>
void main()
{
    printf("Hello World \n");
}
```

Line 1: #include <stdio.h>
As part of compilation, the C compiler runs a program called the C preprocessor. The preprocessor is able to add and remove code from your source file. In this case, the directive #include tells the preprocessor to include code from the file stdio.h. This file contains declarations for functions that the program needs to use. A declaration for the printf function is in this file.

Line 2: void main()
This statement declares the main function. A C program can contain many functions but must always have one main function. A function is a self-contained module of code that can accomplish some task. Functions are examined in a later tutorial. The "void" specifies the return type of main. In this case, nothing is returned to the operating system.

Line 3: {
This opening bracket denotes the start of the program.

Line 4: printf("Hello World \n");
Printf is a function from a standard C library that is used to print strings to the standard output, normally your screen. The compiler links code from these standard libraries to the code you have written to produce the final executable. The "\n" is a special format modifier that tells the printf to put a line feed at the end of the line. If there were another printf in this program, its string would print on the next line.

Line 5: }
This closing bracket denotes the end of the program.

That's it. To get the most of this series of tutorials, you should get access to both a text editor and a C compiler. Some instructions for doing this are in our tutorials on compiling.

This lesson will teach you how to declare and use variables in C. A variable is used to hold data within your program. A variable represents a location in your computer's memory. You can put data into this location and retrieve data out of it. Every variable has two parts, a name and a data type.

Data Types

C provides built in data types for character, float and integer data. A mechanism, using the keyword typedef, exists for creating user-defined types. As an aside, in C you may assign a value to a variable when you declare it.

Integer variables are used to store whole numbers. There are several keywords used to declare integer variables, including int, short, long, unsigned short, unsigned long. The difference deals with the number of bytes used to store the variable in memory, long vs. short, or whether negative and positive numbers may be stored, signed vs. unsigned. These differences will be explained in more advanced tutorials. For now, use int to declare integer variables. On most 32-bit systems, int is synonymous with signed long.

Examples:

```
int count;
int number_of_students = 30;
```

Float variables are used to store floating point numbers. Floating point numbers may contain both a whole and fractional part, for example, 52.7 or 3.33333333. There are several keywords used to declare floating point numbers in C including float, double and long double. The difference here is the number of bytes used to store the variable in memory. Double allows larger values than float. Long double allows even larger values. These differences will be explained in more advanced tutorials. For now, use float for small numbers and double for larger numbers to declare floating point variables.

Examples:

```
float owned = 0.0;
double owed = 1234567.89;
```

Character variables are used to store character values. The use of characters and strings will be covered in a latter tutorial. Character variables are declared with the keyword char.

Examples:

```
char firstInitial = 'J';
char secondInitial = 'K';
```

Variable Names

Valid names can consist of letters, numbers and the underscore, but may not start with a number. A variable name may not be a C keyword such as if, for, else, or while. Variable names are case sensitive. So, Age, AGE, aGE and AgE could be names for different variables, although this is not recommended since it would probably cause confusion and errors in your programs. Note that int, float and double are built in C data types

Constants

This lesson will teach you how to declare and use constants in C. A constant is similar to a variable in the sense that it represents a memory location. It differs in that it cannot be reassigned a new value after initialization. In general, constants are a useful feature that can prevent program bugs and logic errors. Unintended modifications are prevented from occurring. The compiler will catch attempts to reassign new values to constants.

Using #define

There are three techniques used to define constants in C. First, constants may be defined using the preprocessor directive #define. The preprocessor is a program that modifies your source file prior to compilation. Common preprocessor directives are #include, which is used to include additional code into your source file, #define, which is used to define a constant and #if/#endif, which can be used to conditionally determine which parts of your code will be compiled. The #define directive is used as follows.

```
#define pi 3.1415
#define id_no 12345
```

Wherever the constant appears in your source file, the preprocessor replaces it by its value. So, for instance, every "pi" in your source code will be replace by 3.1415. The compiler will only see the value 3.1415 in your code, not "pi". Every "pi" is just replaced by its value. Here is a simple program illustrating the preprocessor directive #define.

```
#include <stdio.h>

#define monday 1
#define tuesday 2
#define wednesday 3
#define thursday 4
#define friday 5
#define saturday 6
#define sunday 7
```

Using const variables

The second technique is to use the keyword const when defining a variable. When used the compiler will catch attempts to modify variables that have been declared const.

```
const float pi = 3.1415;
const int id_no = 12345;
```

There are two main advantages over the first technique. First, the type of the constant is defined. "pi" is float. "id_no" is int. This allows some type checking by the compiler. Second, these constants are variables with a definite scope. The scope of a variable relates to parts of your program in which it is defined. Some variables may exist only in certain functions or in certain blocks of code. You may want to use "id_no" in one function and a completely unrelated "id_no" in your main program. Sorry if this is confusing, the scope of variables will be covered in a latter lesson.

Using enumerations

The third technique to declare constants is called enumeration. An enumeration defines a set of constants. In C an enumerator is of type int. C++ allows variables to be a new enumeration type, which allows the compiler to perform type and bound checking. Unfortunately, C does not provide this. But the enumeration is still useful to define a set of constants instead of using multiple #defines.

```
int main()
{

  enum days {monday=1,tuesday,wednesday,thursday,friday,saturday,sunday};
   enum days today = monday;
}
```

There are a few things to note in this example. First, is clearer to use the enumeration than the #defines used in the earlier example. Next, the variable today is of type int, as are all enumerators. Any expression that can be assigned to an int variable can be assigned to "today". The compiler will not catch bound errors as would occur in C++. For instance, the follow assignment, an error, could appear in the program and not be flagged by the compiler.

Look at the following examples to understand how to assign values to the constants in enumerations.

```
enum COLOR { RED, BLUE, GREEN};
enum SHAPE {SQUARE, RECTANGLE, TRIANGLE, CIRCLE, ELLIPSE};
```

Each enumerated constant (sometimes called an enumerator) has an integer value. Unless specified, the first constant has a value of zero. The values increase by one for each additional constant in the enumeration. So, RED equals 0, BLUE equals 1, and GREEN = 2. SQUARE equals 0, RECTANGLE equals 1, TRIANGLE equals 2 and so forth. The values of each constant can also be specified.

enum SHAPE {SQUARE=5,RECTANGLE,TRIANGLE=17,CIRCLE,ELLIPSE};

Here, SQUARE equals 5, RECTANGLE equals 6, TRIANGLE equals 17, CIRCLE equals 18 and ELLIPSE equals 19.

Input and Output

This lesson covers basic input and output. Usually i/o, input and output, form an important part of any program. To do anything useful your program needs to be able to accept input data and report back your results. In C, the standard library provides routines for input and output. The standard library has functions for i/o that handle input, output, and character and string manipulation. In this lesson, all the input functions described read from standard input and all the output functions described write to standard output. Standard input is usually the keyboard. Standard output is usually the monitor.

Printf returns an integer value. It will either be the number of characters output, or some negative number signifying that an error has occurred. The format is a character string that can contain ordinary characters, which are output unmodified and conversion specifications, which control how the additional arguments are converted to characters for output. The easiest way to understand this is by example.

```
#include <stdio.h>

int main()
{
   int luckyNumber = 5;
   float radius = 2;
   char myName[15] = "Sri";
   char initial = 'R';

   printf("Hello World\n");    /* The format string contains only ordinary
                characters. Ordinary characters are output unmodified.
                A character string in C is of type "char *". */

   printf("My lucky number is %d\n", luckyNumber);    /* The "%d" specifies
                that an integer value will be output. */

   printf("My name is %s\n",myName);    /* The %s specifies that a character
                string will be output. */

   printf("My first initial is %c\n",initial);    /* The %c specifies that a
                character will be output. */

   printf("The area of a circle with radius %f is %f\n", radius, 3.14*radius*radius);
                /* %f specifies that a float will be output. */

   printf("Hello %s or should I say %c\n",myName,initial);
                /* Multiple arguments of different types may be output. */
   return(0);
}
```

Here are the more common conversion specifiers.

Specifier	Argument Type
%d	Int
%f	float or double
%e	float or double, output in scientific notation.
%c	Character
%s	character string (char *)

Scanf returns an integer, either the number of values read in, or EOF if an end of file is reached. EOF is a special termination character, specified in stdio.h, which designates the end of a file. If no values are successfully read, scanf returns 0. To use scanf in a program, the file stdio.h must be included.

```
#include <stdio.h>

int main()
{
   float radius;
      int number;

   printf("Enter the radius of circle: ");
   scanf("%f",&radius);   /* & is the "address of" operator.
                The address of radius is passed into scanf. Passing
                the address of a variable is equivalent to passing
                a pointer containing the address of the variable. */
   printf("A circle of radius %f has area %f\n",radius,3.14*radius*radius);

   printf("Enter a number from 1 to 1000: ");
   scanf("%d",&number);
                /* The address of number is passed to scanf */
   printf("Your number is %d\n",number);
   return(0);
}
```

With one exception scanf will skip over white space such as blanks, tabs and newlines in the input stream. The exception is when trying to read single characters with the conversion specifier %c. In this case, white space is read in. So, it is more difficult to use scanf for single characters. An alternate technique, using getchar, will be described later in this lesson.

.

Conditional Processing

This lesson introduces conditional processing. In previous tutorials, all the code in the examples executed, that is, from the first line of the program to the last, every statement was executed in the order it appeared in the source code. This may be correct for some programs, but others need a way to choose which statements will be executed or run. Conditional processing extends the usefulness of programs by allowing the use of simple logic or tests to determine which blocks of code are executed. In this lesson, a simple guessing game will be developed to illustrate the use of conditional execution.

The if statement is used to conditionally execute a block of code based on whether a test condition is true. If the condition is true the block of code is executed, otherwise it is skipped.

```
#include <stdio.h>

int main()
{

   int number = 5;
   int guess;

   printf("I am thinking of a number between 1 and 10\n");
   printf("Enter your guess, please \n");
   scanf("%d",&guess);
   if (guess == number)
   {
      printf("Incredible, you are correct\n");
   }

   return 0;
}
```

Please try compiling and executing the above. The "==" is called a relational operator. Relational operators, ==, !-, >, >=, <, and <=, are used to compare two operands. The program works, but it needs some improvements. If the user enters 5 as a choice, he gets back a nice message, "Incredible, you are correct". But what happens if the user puts in an incorrect choice? Nothing. No message, no suggestions, nothing. Luckily, for our program user, C has a solution.

The else statement provides a way to execute one block of code if a condition is true, another if it is false.

```
#include <stdio.h>

int main()
{
```

```
   int number = 5;
   int guess;

   printf("I am thinking of a number between 1 and 10\n");
   printf("Enter your guess, please\n");
   scanf("%d",&guess);
   if (guess == number)
   {
      printf("Incredible, you are correct\n");
   }
  else
   {
      printf("Sorry, try again\n");
   }

   return 0;
}
```

This lesson introduces conditional processing. In previous tutorials, all the code in the examples executed, that is, from the first line of the program to the last, every statement was executed in the order it appeared in the source code. This may be correct for some programs, but others need a way to choose which statements will be executed or run. Conditional processing extends the usefulness of programs by allowing the use of simple logic or tests to determine which blocks of code are executed. In this lesson, a simple guessing game will be developed to illustrate the use of conditional execution.

The if statement is used to conditionally execute a block of code based on whether a test condition is true. If the condition is true the block of code is executed, otherwise it is skipped.

```
#include <stdio.h>

int main()
{

   int number = 5;
   int guess;

   printf("I am thinking of a number between 1 and 10\n");
   printf("Enter your guess, please \n");
   scanf("%d",&guess);
   if (guess == number)
   {
      printf("Incredible, you are correct\n");
   }
```

```
    return 0;
}
```

Please try compiling and executing the above. The "==" is called a relational operator. Relational operators, ==, !-, >, >=, <, and <=, are used to compare two operands. The program works, but it needs some improvements. If the user enters 5 as a choice, he gets back a nice message, "Incredible, you are correct". But what happens if the user puts in an incorrect choice? Nothing. No message, no suggestions, nothing. Luckily, for our program user, C has a solution.

The else statement provides a way to execute one block of code if a condition is true, another if it is false.

```
#include <stdio.h>

int main()
{

    int number = 5;
    int guess;

    printf("I am thinking of a number between 1 and 10\n");
    printf("Enter your guess, please\n");
    scanf("%d",&guess);
    if (guess == number)
    {
        printf("Incredible, you are correct\n");
    }
   else
    {
        printf("Sorry, try again\n");
    }

    return 0;
}
```

Summary of Relational Operators

Operator	Description	Example	Evaluation
==	equal	5 == 4	FALSE
		5 == 5	TRUE
!=	not equal	5 != 4	TRUE

		5 != 5	FALSE
>	greater than	5 > 4	TRUE
		5 > 5	FALSE
>=	greater than or equal	5 >= 4	TRUE
		5 >= 5	TRUE
<	less than	5 < 4	FALSE
		5 < 5	FALSE
<=	less than or equal	5 <= 4	FALSE
		5 <= 5	TRUE

This second lesson on conditional processing introduces both the switch statement and logical operators. The switch statement is a construct that is used to replace deeply nested or chained if/else statements. Nested if/else statements arise when there are multiple alternative threads of execution based on some condition. Here's an example. Suppose that an ice cream store has asked us to write a program that will automate the taking of orders. We will need to present a menu and then based on the customer's choice take an appropriate action.

```
#include <stdio.h>

int main()
{
   int choice;

   printf("What flavor ice cream do want?\n");
   printf("Enter 1 for chocolate\n");
   printf("Enter 2 for vanilla\n");
   printf("Enter 3 for strawberry\n");
   printf("Enter 4 for green tea flavor, yuck\n");
   printf("Enter you choice: ");

   scanf("%d",&choice);

  if (choice == 1) {
      printf("Chocolate, good choice\n");
   }
   else if (choice == 2) {
      printf("Vanillarific\n");
   }
   else if (choice == 3) {
      printf("Berry Good\n");
   }
   else if (choice == 4) {
      printf("Big Mistake\n");
   }
   else {
     printf("We don't have any\n");
```

```
      printf("Make another selection \n");
   }

   return 0;
}
```

This program will work fine, but the if/else block is cumbersome. It would be easy, particularly if there were more choices and maybe sub choices involving more if/else's to end up with program that doesn't perform the actions intended. Here's the same program with a switch.

```
#include <stdio.h>

int main()
{
   int choice;

   printf("What flavor ice cream do want?\n");
   printf("Enter 1 for chocolate\n");
   printf("Enter 2 for vanilla\n");
   printf("Enter 3 for strawberry\n");
   printf("Enter 4 for green tea flavor, yuck\n");
   printf("Enter you choice: \n");

   scanf("%d",&choice);

   switch (choice) {
   case 1:
      printf("Chocolate, good choice\n");
      break;
   case 2:
      printf("Vanillarific\n");
      break;
   case 3:
      printf("Berry Good\n");
      break;
   case 4:
      printf("Big Mistake\n");
      break;
   default:
      printf("We don't have any\n");
      printf("Make another selection\n");
   }

   return 0;
}
```

The general form of a switch statement is:

```
switch (variable)
 {
   case expression1:
      do something 1;
      break;
   case expression2:
      do something 2;
      break;
     ....
   default:
      do default processing;
}
```

Each expression must be a constant. The variable is compared for equality against each expression. It is not possible to use the other relational operators discussed in the last lesson or the logical operators that will be introduced latter in this lesson. When an expression is found that is equal to the tested variable, execution continues until a break statement is encountered. It is possible to have a case without a break. This causes execution to fall through into the next case. This is sometimes very useful. Suppose we need to determine if a letter stored in a variable is a vowel or consonant.

```
switch (myLetter) {
   case 'A':
   case 'E':
   case 'I':
   case 'O':
   case 'U':
      vowelCnt++; /* increments vowel count */
         /* same as, vowelCnt = vowelCnt + 1; */
      break;
   default:
      consonantCnt = consonantCnt + 1;
}
```

If any vowels are found execution drops through each case until the count is incremented and the "break" is encountered. For some coding practice, how could the same logic be implemented using if and else's?
In more realistic examples, it is probably necessary to evaluate multiple conditions to determine what parts of code should execute. For instance, if condition 1 is true and condition 2 is true process one way, if condition 1 is true and condition two is false process another way. C provides several logical operators that allow more complex relational expressions to be formed and evaluated.

Logical Operators

Operator	Description	Example	Evaluation
&&	AND	(5 > 3) AND (5 > 6)	FALSE
&&	AND	(5 > 3) AND (5 > 4)	TRUE

||	OR	(5 > 3) OR (5 > 6)	TRUE
||	OR	(5 > 3) OR (5 > 4)	TRUE
!	NOT	!(5 > 3)	FALSE
!	NOT	!(5 > 6)	TRUE

As can be seen in this table, && will return true only if both expressions are true, while || will be true if either expression is true. The operator "!" provides logical negation. One very important consideration when forming expressions is the order of precedence of the relational and logical operators. Relational operators are of higher precedence than the logical and the order of evaluation is from left to right. Here are some examples that illustrate what this means.

if (myChoice == 'A' and myAge < 25) is evaluated as
if ((myChoice == 'A') and (myAge < 25))

Suppose x = 8, y = 49, z = 1.
if (x < 7 && y > 50 || z < 2) is evaluated as
if (((x < 7) && (y > 50)) || (z < 2)) which is TRUE, not as
if ((x < 7) && ((y > 50) || (z < 2)) which is FALSE.

Even if you are sure about the order of precedence of an expression, use explicit parenthesis. This serves to increase readability and will help avoid errors.

Looping

This lesson covers three constructs that are used to create loops in C programs. Loops can be created to execute a block of code for a fixed number of times. Alternatively, loops can be created to repetitively execute a block of code until a boolean condition changes state. For instance, the loop may continue until a condition changes from false to true, or from true to false. In this case, the block of code being executed must update the condition being tested in order for the loop to terminate at some point. If the test condition is not modified somehow within the loop, the loop will never terminate. This creates a programming bug known as an infinite loop.

While

The while loop is used to execute a block of code as long as some condition is true. If the condition is false from the start the block of code is not executed at all. Its syntax is as follows.

*while (*tested condition is satisfied*) {*
  block of code
}

Here is a simple example of the use of while. This program counts from 1 to 100.

```
#include <stdio.h>

int main()
{
   int count = 1;

   while (count <= 100)
   {
      printf("%d\n",count);
      count += 1; /* Shorthand for count = count + 1 */
   }

   return 0;
}
```

Here is a more realistic use of while. This program determines the minimum number of bits needed to store a positive integer. The largest unsigned number that can be stored in N bits is $(2^N - 1)$.

```
#include <stdio.h>

int main()
{
   int bitsRequired = 1; //the power of 2
   int largest = 1; //largest number that can be stored
   int powerOf2 = 2;
   int number;

   printf("Enter a positive integer: ");
   scanf("%d",&number);

   while (number > largest)
   {
      bitsRequired += 1; //Shorthand for bitsRequired = bitsRequired + 1
      powerOf2 = powerOf2 * 2;
      largest = powerOf2 - 1;
   }

   printf("To store %d requires %d bits",number,bitsRequired);

   return 0;
}
```

Do

The do loop also executes a block of code as long as a condition is satisfied. The difference between a "do" loop and a "while" loop is that the while loop tests its condition at the top of its loop; the "do" loop tests its condition at the bottom of its loop. As noted above, if the test condition is false as the while loop is entered the

block of code is skipped. Since the condition is tested at the bottom of a do loop, its block of code is always executed at least once. The "do" loops syntax is as follows

```
do {
   block of code
} while (condition is satisfied)
```

Here is an example of the use of a do loop. The following program is a game that allows a user to guess a number between 1 and 100. A "do" loop is appropriate since we know that winning the game always requires at least one guess.

```
#include <stdio.h>

int main()
{

   int number = 44;
   int guess;

   printf("Guess a number between 1 and 100\n");
   do {
      printf("Enter your guess: ");
      scanf("%d",&guess);
      if (guess > number) {
         printf("Too high\n");
      }
      if (guess < number) {
         printf("Too low\n");
      }
   } while (guess != number);
   printf("You win. The answer is %d",number);
   return 0;
}
```

For

The third looping construct in C is the for loop. The for loop can execute a block of code for a fixed number of repetitions. Its syntax is as follows.

```
for (initializations;test conditions;actions)
{
   block of code
}
```

The simplest way to understand for loops is to study several examples.
First, here is a for loop that counts from 1 to 10.

```
for (count = 1; count <= 10; count++)
{
```

```
   printf("%d\n",count);
}
```

The test conditions may be unrelated to the variables being initialized and updated (assigned). Here is a loop that counts until a user response terminates the loop.

```
for (count = 1; response != 'N'; count++)
{
   printf("%d\n",count);
   printf("Continue (Y/N): \n");
   scanf("%c",&response);
}
```

More complicated test conditions are also allowed. Suppose the user of the last example never enters "N", but the loop should terminate when 100 is reached, regardless.

```
for (count = 1; (response != 'N') && (count <= 100); count++)
{
   printf("%d\n",count);
   printf("Continue (Y/N): \n");
   scanf("%c",&response);
}
```

It is also possible to have multiple initializations and multiple actions. This loop starts one counter at 0 and another at 100, and finds a midpoint between them.

```
for (i = 0, j = 100; j != i; i++, j--)
{
   printf("i = %d, j = %d\n",i,j);
}
printf("i = %d, j = %d\n",i,j);
```

Initializations are optional. For instance, suppose we need to count from a user specified number to 100. The first semicolon is still required as a place keeper.

```
printf("Enter a number to start the count: ");
scanf("%d",&count);
for ( ; count < 100 ; count++)
{
   printf("%d\n",count);
}
```

Jump statements

break

Using break we can leave a loop even if the condition for its end is not fulfilled. It can be used to end an infinite loop, or to force it to end before its natural end. For example, we are going to stop the count down before its natural end (maybe because of an engine check failure?):

```
// break loop example

#include <iostream>
using namespace std;

int main ()
{
  int n;
  for (n=10; n>0; n--)
  {
    cout << n << ", ";
    if (n==3)
    {
      cout << "countdown aborted!";
      break;
    }
  }
  return 0;
}
```

continue

The continue statement causes the program to skip the rest of the loop in the current iteration as if the end of the statement block had been reached, causing it to jump to the start of the following iteration. For example, we are going to skip the number 5 in our countdown:

```
// continue loop example
#include <iostream>
using namespace std;

int main ()
{
  for (int n=10; n>0; n--) {
    if (n==5) continue;
    cout << n << ", ";
  }
  cout << "FIRE!";
  return 0;
}
```

goto

goto allows to make an absolute jump to another point in the program. You should use this feature with caution since its execution causes an unconditional jump ignoring any type of nesting limitations.
The destination point is identified by a label, which is then used as an argument for the goto statement. A label is made of a valid identifier followed by a colon (:).

Generally speaking, this instruction has no concrete uses in structured or object oriented programming aside from those that low-level programming fans may find for it. For example, here is our countdown loop using goto:

```
// goto loop example

#include <iostream>
using namespace std;

int main ()
{
  int n=10;
  loop:
  cout << n << ", ";
  n--;
  if (n>0) goto loop;
  cout << "FIRE!";
  return 0;
}
```

Practice For practice, try implementing programs that will count down from 10 to 1 using while, do and for loops.

Arrays

This lesson introduces arrays. Arrays are a data structure that is used to store a group of objects of the same type sequentially in memory. All the elements of an array must be the same data type, for example float, char, int, pointer to float, pointer to int, a structure or function. Structures provide a way to organize related data and will be studied in a later lesson. Functions provide a way to define a new operation. They are used to calculate a result or update parameters. Functions will be covered in a later lesson. The elements of an array are stored sequentially in memory. This allows convenient and powerful manipulation of array elements using pointers.

Defining Arrays
An array is defined with this syntax.

datatype arrayName[size];

Examples:

```
int ID[30];
        /* Could be used to store the ID numbers of students in a class */

float temperatures[31];
        /* Could be used to store the daily temperatures in a month */

char name[20];
        /* Could be used to store a character string.
        Character strings in C are terminated by the null character, '\0'.
        This will be discussed later in the this lesson. */

int *ptrs[10];
        /* An array holding 10 pointers to integer data */

unsigned short int[52];
        /* Holds 52 unsigned short integer values */
```

Using Arrays

Arrays in C are zero based. Suppose an array named myExample contains N elements. This array is indexed from 0 to (N-1). The first element of myExample is at index 0 and is accessed as myExample[0]. The second element is at index 1 and is accessed as myExample[1]. The last element is at index (N-1) and is accessed as myExample[N-1]. As a concrete example, suppose N equals 5 and that myExample will store integer data.

```
int myExample[5];     /* Defines myExample to be of length 5 and to contain integer data */

myExample[0]    /* First element of myExample */
myExample[1]    /* Second element of myExample */
myExample[2]    /* Third element of myExample */
myExample[3]    /* Fourth element of myExample */
myExample[4]    /* Fifth and final element of myExample */
```

Here is a sample program that calculates and stores the squares of the first one hundred positive integers.

```
#include <stdio.h>

int main()
{
```

```
   int square[100];
   int i;    /* loop index */;
   int k;    /* the integer */

   /* Calculate the squares */
   for (i = 0; i < 100; i++) {
      k = i + 1;    /* i runs from 0 to 99 */
                    /* k runs from 1 to 100 */
      square[i] = k*k;
      printf("The square of %d is %d\n",k,square[i]);
   }

   return 0;
}
```

Practice Problem 1: A Fibonacci sequence is a numerical sequence such that after the second element all numbers are equal to the sum of the previous two elements.

1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, ...

Write a program to calculate and print the first 20 elements of this sequence.

Strings

Strings are nothing but character arrays, i.e an array of datatype char. It is similar to that of an integer array in declaring and usage. But it also can be used in a easier way

For example to input a character array

```
char s[80];
scanf("%s",&s);
```

will suffice, this will take in the whole of the array, there is no need to use a for loop in case of a string. But by taking input using scanf it is not possible to input white spaces (' ').the string willterminate on encountering a white space, and in case of a string it is not necessary to inpu t all the 80 elements of the array. Suppose you want to enter only NIT as the input, then the compiler will take the input and when u press the enter key to signify the end of the string, it will automatically place a special character called NULL(\0) in the array. This NULL character signifies the end of the string. Similarly to print it out:

printf("%s",s);

is sufficient. This statement will print out all the characters till the NULL terminator is reached. An example:

```
#include <stdio.h>

int main()
{
   char sentence[] = "Hello World";
   int count = 0;
   int i;

   for (i = 0; sentence[i] != '\0'; i++)
   {
      count++;
   }
   printf("The string %s has %d characters ",
      sentence,count);
   printf("and is stored in %d bytes\n",
      count+1);

   return 0;
}
```

There are some other build in functions to make the usage of string easier.

Functions for Input and Output

getchar
getchar reads a single character from standard input. Its prototype is:

```
int getchar();
```

It returns int rather than char because the "end of file", EOF, character must be handled. EOF is an int and is too large to fit in a char variable. A newline character in C is '\n'. This designates the end of a line.

putchar
putchar writes a single character to standard output. Its prototype is:
int putchar(int value);

Here is a simple program which echoes back everything that is typed in. Note that depending on which operating system you are using, EOF is entered by typing control-z or control-d. So, if you try running this code, type control-z or control-d to end execution

```
#include <stdio.h>

int main()
{
   int c;
```

```
   while ((c = getchar()) != EOF)
   {
      putchar(c);
   }

   return(0);
}
```

gets
gets reads a line of input into a character array. Its prototype is:

*char *gets(char *buffer);*

It returns a pointer to the character string if successful, or NULL if end of file is reached or if an error has occurred. The string is also read into the character array specified as an argument. The character string will be terminated be a "\0", which is standard for C.

Other Useful Standard Library Functions for Input and Output, Continued

puts
puts writes a line of output to standard output. Its prototype is:

*int puts(const char *buffer);*

It terminates the line with a newline, '\n'. It will return EOF is an error occurred. It will return a positive number on success.

Here is the "echo" program implemented using the functions gets and puts. If you run this, remember to type control-z or control-d to end execution.

```
#include <stdio.h>

int main()
{
   char buffer[120]; /* Holds input and output strings */
   char *pt; /* A pointer to datatype char */
   int returnCode;
   while ((pt = gets(buffer)) != NULL)
   {
      returnCode = puts(buffer);
      if (returnCode == EOF)
      {
         printf("Error on output\n");
   }
   }
```

```
    return(0);
}
```

There are some very important things to note in the this example. First, 'gets' returns a NULL pointer when EOF is reached. This is the condition that must be checked in while loop. Second, puts returns EOF on failure. A check is made on this, and an appropriate error message is output if an error has occurred. Finally, the character array buffer is declared to be of length 120. The function gets makes no check on the array size to see if it is large enough to hold the string entered. It is up to the programmer to create a character array that is large enough. If the entered string is larger than the array, gets will still put the string in memory starting at the location of the first byte of the designated array. Since the string is longer than the array, the result is that memory past the end of the array will be overwritten with the entered string. This will cause either erratic behavior or failure (core dump) of the program. This weakness in the C language can lead to program bugs. On the other hand, the low level access to memory and data can be of great utility and is a strength of C and C++ that is lacking in some other languages. C++ allows C style handling of strings but also provides a string class with more sophisticated string handling to alleviate the need for the programmer to address this issue.

Some other concepts regarding strings and its usage are dealt under the pointers chapter.

Practice
1) Try compiling and running some of the examples presented in this lesson.
2) Write a program to read in the name and address of the user and echo back that information.
Solution
3) Write a program to prompt a user for a nickname, the year they were born as an integer, and their weight in pounds as a float. Calculate their age. Calculate their weight in kilograms. Write this information to standard output. Note there are 2.2046 pounds per kilogram

Pointers

Pointers are variables that hold addresses in C and C++. They provide much power and utility for the programmer to access and manipulate data in ways not seen in some other languages. They are also useful for passing parameters into functions in a manner that allows a function to modify and return values to the calling routine. When used incorrectly, they also are a frequent source of both program bugs and programmer frustration.

Introduction
As a program is executing all variables are stored in memory, each at its own unique address or location. Typically, a variable and its associated memory address contain data values. For instance, when you declare:

```
int count = 5;
```

The value "5" is stored in memory and can be accessed by using the variable "count". A pointer is a special type of variable that contains a memory address rather than a data value. Just as data is modified when a normal variable is used, the value of the address stored in a pointer is modified as a pointer variable is manipulated.

Usually, the address stored in the pointer is the address of some other variable.

```
int *ptr;
ptr = &count /* Stores the address of count in ptr */
   /* The unary operator & returns the address of a variable */
```

To get the value that is stored at the memory location in the pointer it is necessary to dereference the pointer. Dereferencing is done with the unary operator "*".

```
int total;
total = *ptr;
   /* The value in the address stored in ptr is assigned to total */
```

The best way to learn how to use pointers is by example. There are examples of the types of operations already discussed below. Pointers are a difficult topic. Don't worry if everything isn't clear yet.

Declaration and Initialization
Declaring and initializing pointers is fairly easy.

```
int main()
{
   int j;
   int k;
   int l;
   int *pt1;    /* Declares an integer pointer */
   int *pt2;    /* Declares an integer pointer */
   float values[100];
   float results[100];
   float *pt3;    /* Declares a float pointer */
   float *pt4;    /* Declares a float pointer */

   j = 1;
   k = 2;
   pt1 = &j;    /* pt1 contains the address of the variable j */
   pt2 = &k;    /* pt2 contains the address of variable k */
   pt3 = values;
      /* pt3 contains the address of the first element of values */
   pt3 = &values[0];
      /* This is the equivalent of the above statement */
```

```
   return 0;
}
```

Pointer Dereferencing/Value Assignment
Dereferencing allows manipulation of the data contained at the memory address stored in the pointer. The pointer stores a memory address. Dereferencing allows the data at that memory address to be modified. The unary operator "*" is used to dereference.
For instance:

```
*pt1 =*pt1 + 2;
```

This adds two to the value "pointer to" by pt1. That is, this statement adds 2 to the contents of the memory address contained in the pointer pt1. So, from the main program, pt1 contains the address of j. The variable "j" was initialized to 1. The effect of the above statement is to add 2 to j.

The contents of the address contained in a pointer may be assigned to another pointer or to a variable.

```
*pt2 = *pt1;
   /* assigns the contents of the memory pointed to by pt1 */
   /* to the contents of the memory pointer to by pt2; */
k = *pt2;
   /* assigns the contents of the address pointer to by pt2 to k. */
```

Pointer Arithmetic
Part of the power of pointers comes from the ability to perform arithmetic on the pointers themselves. Pointers can be incremented, decremented and manipulated using arithmetic expressions. Recall the float pointer "pt3" and the float array "values" declared above in the main program.

```
pt3 = &values[0];
   /* The address of the first element of "values" is stored in pt3*/
pt3++;
   /* pt3 now contains the address of the second element of values */
*pt3 = 3.1415927;
   /* The second element of values now has pie (actually pi)*/
pt3 += 25;
   /* pt3 now points to the 27th element of values */
*pt3 = 2.22222;
   / The 27th element of values is now 2.22222 */

pt3 = values;
   /*pt3 points to the start of values, now */

for (ii = 0; ii < 100; ii++)
{
    *pt3++ = 37.0;    /* This sets the entire array to 37.0 */
```

```
}

pt3 = &values[0];
   /* pt3 contains the address of the first element of values */
pt4 = &results[0];
   /* pt4 contains the address of the first element of results */

for (ii=0; ii < 100; ii++)
{
   *pt4 = *pt3;
      /* The contents of the address contained in pt3 are assigned to
      the contents of the address contained in pt4 */
    pt4++;
    pt3++;
}
```

Relationship Between Pointers and Arrays

In some cases, a pointer can be used as a convenient way to access or manipulate the data in an array. Please see Lesson 8 if you need a refresher on pointers.

Suppose the following declarations are made.

```
float temperatures[31];
   /* An array of 31 float values, the daily temperatures in a month */

float *temp;    /* A pointer to type float */
```

Since temp is a float pointer, it can hold the address of a float variable. The address of the first element of the array temperatures can be assigned to temp in two ways.

```
temp = &temperatures[0];
temp = temperatures;
   /* This is an alternate notation for the first
   element of the array. Same as temperatures = &temperatures[0]. */
```

The temperature of the first day can be assigned in two ways.

```
temperatures[0] = 29.3;    /* brrrrrrr */
*temp = 15.2;    /* BRRRRRRR */
```

Other elements can be updated via the pointer, as well.

```
temp = &temperatures[0];

*(temp + 1) = 19.0;
   /* Assigns 19.0 to the second element of temperatures */
```

```
temp = temp + 9;
   /* temp now has the address of the 10th element of the array */

*temp = 25.0;
   /* temperatures[9] = 25, remember that arrays are zero based,
   so the tenth elementis at index 9. */

temp++;    /* temp now points at the 11th element */

*temp = 40.9;    /* temperatures[10] = 40.9 */
```

Pointers are particularly useful for manipulating strings, which are stored as null terminated character arrays in C

Character Arrays

Strings are stored in C as character arrays terminated by the null character, '\0'. The array length must be at least one greater than the length of the string to allow storage of the terminator. String constants or literals are stored internally as a null character terminated character array.

Assigning a character literal to an array is done as follows.

```
char str1[] = "Hello World";
char str2[] = "Goodbye World";
```

The compiler automatically sizes the arrays correctly. For this example, str1 is of length 12, str2 is of length 14. These lengths include space for the null character that is added at the end of the string.

A character pointer can also be assigned the address of a string constant or of a character array.

```
char *lpointer = "Hello World";
   /* Assigns the address of the literal to lpointer */

char *apointer = str1;
   /* Assigns the starting address of str1 to apointer */

char *apointer = &str1[0];
   /* Assigns the starting address of str1 to apointer */
```

There is no direct means in the C language to copy one array to another, or one string to another. It must be done either with a standard library function or element wise in a loop. Let's try to copy on string to another.

```
#include <stdio.h>
```

```
int main()
{
   char str1[] = "Hello World";
   char str2[] = "Goodbye World";

   str2 = str1;

   return 0;
}
```

Can you see what's wrong with this code. As stated, there is no operation to assign one array to another in C. This code produced this compiler error.

error: '=' : cannot convert from 'char [12]' to 'char [14]'
There is no context in which this conversion is possible.

Character Arrays, Continued
Now let's make another attempt using character pointers.

```
#include <stdio.h>

int main()
{
   char str1[] = "Hello World";
   char str2[] = "Goodbye World";
   char *cpt1;
   char *cpt2;

   cpt1 = &str1[0];
   cpt2 = &str2[0];

   printf("str1 is %s\n",str1);
   printf("str2 is %s\n",str2);
   printf("cpt1 is %s\n",cpt1);
   printf("cpt2 is %s\n",cpt2);

   cpt2 = cpt1;

   printf("str1 is %s\n",str1);
   printf("str2 is %s\n",str2);
   printf("cpt1 is %s\n",cpt1);
   printf("cpt2 is %s\n",cpt2);

   return 0;
}
```

Results:
str1 is Hello World
str2 is Goodbye World
cpt1 is Hello World
cpt2 is Goodbye World
str1 is Hello World
str2 is Goodbye World
cpt1 is Hello World
cpt2 is Hello World

As can be seen from the results, all that happened is that the pointer cpt2 was assigned the value of cpt1, that is, the address of str1. The contents of the array str2 were not changed. The only way to copy a string or any array in C is element by element. Here is a program that correctly copies on string to another.

```
#include <stdio.h>

int main()
{
   int i;
   char str1[] = "Hello World";
   char str2[] = "Goodbye World";

   printf("str1 is %s\n",str1);
   printf("str2 is %s\n",str2);

   i = 0;
   while ((str2[i] = str1[i]) != '\0') {
      i++;
   }

   printf("str1 is %s\n",str1);
   printf("str2 is %s\n",str2);

   return 0;
}
```

Results:
str1 is Hello World
str2 is Goodbye World
str1 is Hello World
str2 is Hello World

Practice Problem 2: Try reimplementing the above program using pointers in the copy loop.
Hints:

```
cpt1 = &str1[0];
```

```
cpt2 = &str2[0];
Use these pointers in the while loop, remember to dereference.
```

Multidimensional Arrays

The C language also allows multidimensional arrays. These arrays are like matrices. They are defined as follows.

```
float temperatures[12][31];    /* Used to store temperature data for a year */

float altitude[100][100];    /* Used to store the altitudes of 10000 grid points
        of a 100 by 100 mile square */
```

A common way to access the elements of a multidimensional arrays is with nested for loops.

```
#define MAXI 50
#define MAXJ 75

int i;
int j;
float values[MAXI][MAXJ];

for (i = 0; i < MAXI; i++) {
   for (j = 0; j < MAXJ; j++) {
      values[i][j] = whatever;
   }
}
```

Important Note about Array Dimensions

The C language performs no error checking on array bounds. If you define an array with 50 elements and you attempt to access element 50 (the 51st element), or any out of bounds index, the compiler issues no warnings. It is the programmer's task alone to check that all attempts to access or write to arrays are done only at valid array indexes. Writing or reading past the end of arrays is a common programming bug and can be hard to isolate.

What will happen if a program accesses past the end of an array? Suppose a program has the following code.

```
int val;
int buffer[10];
val = buffer[10];
   /* Bug, remember that the indexes of buffer run from 0 to 9. */
```

What value will be in val? Whatever happens to be in memory at the location right after the

end of the array. This value could be anything. Worse yet, the program may continue to run with the incorrect value and no warnings are issued.
What will happen if a program writes past the end of an array? Suppose a program has the following code.

```
int buffer[10];
buffer[593] = 99;
```

The value of 99 will be written at the memory location, buffer + 593. "buffer" is a pointer to the beginning of the array. buffer + 593 is pointer arithmetic for the address equal to the starting address of the array plus the size of 593 integers. The overwriting of the value at this memory location will change the value of whatever variable is stored there. Some other variable may have its value changed unintentionally. If the program writes unintentionally to memory locations that not valid, the program may crash.
The most common cause of writing/reading to invalid array indexes are errors in loop limits.

```
int i;
float b[10];
for (i < 0 ; i <= 10; i++) {
   b[i] = 3.14 * i * i;
}
```

This loop should use "<" rather than "<="

Lets start with CPP

Standard Output (cout)

By default, the standard output of a program is the screen, and the C++ stream object defined to access it is cout.

cout is used in conjunction with the insertion operator, which is written as << (two "less than" signs).

```
cout << "Output sentence"; // prints Output sentence on screen
cout << 120;              // prints number 120 on screen
cout << x;                // prints the content of x on screen
```

The << operator inserts the data that follows it into the stream preceding it. In the examples above it inserted the constant string Output sentence, the numerical constant 120 and variable x into the standard output stream cout. Notice that the sentence in the first instruction is enclosed between double quotes (") because it is a constant string of characters. Whenever we want to use constant strings of characters we must enclose them between double quotes (") so that they can be clearly distinguished from variable names. For example, these two sentences have very different results:

```
cout << "Hello";  // prints Hello
cout << Hello;    // prints the content of Hello variable
```

The insertion operator (<<) may be used more than once in a single statement:

```
cout << "Hello, " << "I am " << "a C++ statement";
```

This last statement would print the message Hello, I am a C++ statement on the screen. The utility of repeating the insertion operator (<<) is demonstrated when we want to print out a combination of variables and constants or more than one variable:

```
cout << "Hello, I am " << age << " years old and my zipcode is " << zipcode;
```

If we assume the age variable to contain the value 24 and the zipcode variable to contain 90064 the output of the previous statement would be:

```
Hello, I am 24 years old and my zipcode is 90064
```

It is important to notice that cout does not add a line break after its output unless we explicitly indicate it, therefore, the following statements:

```
cout << "This is a sentence.";
cout << "This is another sentence.";
```

will be shown on the screen one following the other without any line break between them:

This is a sentence.This is another sentence.

even though we had written them in two different insertions into cout. In order to perform a line break on the output we must explicitly insert a new-line character into cout. In C++ a new-line character can be specified as \n (backslash, n):

```
cout << "First sentence.\n ";
cout << "Second sentence.\nThird sentence.";
```

This produces the following output:

First sentence.
Second sentence.
Third sentence.

Additionally, to add a new-line, you may also use the endl manipulator. For example:

```
cout << "First sentence." << endl;
cout << "Second sentence." << endl;
```

would print out:

First sentence.
Second sentence.

The endl manipulator produces a newline character, exactly as the insertion of '\n' does, but it also has an additional behavior when it is used with buffered streams: the buffer is flushed. Anyway, cout will be an unbuffered stream in most cases, so you can generally use both the \n escape character and the endl manipulator in order to specify a new line without any difference in its behavior.

Standard Input (cin).

The standard input device is usually the keyboard. Handling the standard input in C++ is done by applying the overloaded operator of extraction (>>) on the cin stream. The operator must be followed by the variable that will store the data that is going to be extracted from the stream. For example:

```
int age;
cin >> age;
```

The first statement declares a variable of type int called age, and the second one waits for an input from cin (the keyboard) in order to store it in this integer variable.

cin can only process the input from the keyboard once the RETURN key has been pressed. Therefore, even if you request a single character, the extraction from cin will not process the input until the user presses RETURN after the character has been introduced.

You must always consider the type of the variable that you are using as a container with cin extractions. If you request an integer you will get an integer, if you request a character you will get a character and if you request a string of characters you will get a string of characters.

```
// i/o example

#include <iostream>
using namespace std;

int main ()
{
  int i;
  cout << "Please enter an integer value: ";
  cin >> i;
  cout << "The value you entered is " << i;
  cout << " and its double is " << i*2 << "\n";
  return 0;
}
```

Would print out:

Please enter an integer value: 15

The value you entered is 15 and its double is 30

The user of a program may be one of the factors that generate errors even in the simplest programs that use cin (like the one we have just seen). Since if you request an integer value and the user introduces a name (which generally is a string of characters), the result may cause your program to misoperate since it is not what we were expecting from the user. So when you use the data input provided by cin extractions you will have to trust that the user of your program will be cooperative and that he/she will not introduce his/her name or something similar when an interger value is requested. A little ahead, when we see the stringstream class

We will see a possible solution for the errors that can be caused by this type of user input.

You can also use cin to request more than one datum input from the user:

```
cin >> a >> b;
```

is equivalent to:

```
cin >> a;
cin >> b;
```

In both cases the user must give two data, one for variable a and another one for variable b that may be separated by any valid blank separator: a space, a tab character or a newline.

cin and strings

We can use cin to get strings with the extraction operator (>>) as we do with fundamental data type variables:

```
cin >> mystring;
```

However, as it has been said, cin extraction stops reading as soon as if finds any blank space character, so in this case we will be able to get just one word for each extraction. This behavior may or may not be what we want; for example if we want to get a sentence from the user, this extraction operation would not be useful.

In order to get entire lines, we can use the function getline, which is the more recommendable way to get user input with cin:

```
// cin with strings
#include <iostream>
#include <string>
using namespace std;

int main ()
{
  string mystr;
  cout << "What's your name? ";
  getline (cin, mystr);
  cout << "Hello " << mystr << "\n";
  cout << "What is your favorite team? ";
  getline (cin, mystr);
  cout << "\nI like " << mystr << " too!\n";
  return 0;
}
```

Would print out:

What's your name? Juan Soulié
Hello Juan Soulié.
What is your favorite team? LA Lakers

I like LA Lakers too!

Notice how in both calls to getline we used the same string identifier (mystr). What the program does in the second call is simply to replace the previous content by the new one that is introduced.

Functions

Using functions we can structure our programs in a more modular way, accessing all the potential that structured programming can offer to us in C++.

A function is a group of statements that is executed when it is called from some point of the program. The following is its format:

type name (parameter1, parameter2, ...) { statement }

where:

- type is the data type specifier of the data returned by the function (optional).
- name is the identifier by which it will be possible to call the function.
- parameters (as many as needed): Each parameter consists of a data type specifier followed by an identifier, like any regular variable declaration (for example: int x) and which acts within the function as a regular local variable. They allow to pass arguments to the function when it is called. The different parameters are separated by commas.
- statements is the function's body. It is a block of statements surrounded by braces { }.
- Here you have the first function example:

```
// function example
#include <iostream>
using namespace std;
```

```
int addition (int a, int b)
{
  int r;
  r=a+b;
  return (r);
}

int main ()
{
  int z;
  z = addition (5,3);
  cout << "The result is " << z;
  return 0;
}
```

Would print out:
The result is 8

In order to examine this code, first of all remember something said at the beginning of this tutorial: a C++ program always begins its execution by the main function. So we will begin there.

We can see how the main function begins by declaring the variable z of type int. Right after that, we see a call to a function called addition. Paying attention we will be able to see the similarity between the structure of the call to the function and the declaration of the function itself some code lines above:

```
int addition (int a, int b)
                 ↑      ↑
z = addition (   5   ,   3   );
```

The parameters and arguments have a clear correspondence. Within the main function we called to addition passing two values: 5 and 3, that correspond to the int a and int b parameters declared for function addition.

At the point at which the function is called from within main, the control is lost by main and passed to function addition. The value of both arguments passed in the call (5 and 3) are copied to the local variables int a and int b within the function.

Function addition declares another local variable (int r), and by means of the expression r=a+b, it assigns to r the result of a plus b. Because the actual parameters passed for a and b are 5 and 3 respectively, the result is 8.

The following line of code:

```
return (r);
```

finalizes function addition, and returns the control back to the function that called it in the first place (in this case, main). At this moment the program follows it regular course from the same point at which it was interrupted by the call to addition. But additionally, because the return statement in function addition specified a value: the content of variable r (return (r);), which at that moment had a value of 8. This value becomes the value of evaluating the function call.

```
int addition (int a, int b)
 |8
 v
z = addition (  5  ,  3  );
```

So being the value returned by a function the value given to the function call itself when it is evaluated, the variable z will be set to the value returned by addition (5, 3), that is 8. To explain it another way, you can imagine that the call to a function (addition (5,3)) is literally replaced by the value it returns (8).

The following line of code in main is:

```
Cout << "The result is " << z;
```

That, as you may already expect, produces the printing of the result on the screen.

And here is another example about functions:

```
// function example
#include <iostream>
using namespace std;

int subtraction (int a, int b)
{
  int r;
  r=a-b;
  return (r);
}

int main ()
{
  int x=5, y=3, z;
  z = subtraction (7,2);
  cout << "The first result is " << z << '\n';
  cout << "The second result is " << subtraction (7,2) << '\n';
  cout << "The third result is " << subtraction (x,y) << '\n';
  z= 4 + subtraction (x,y);
  cout << "The fourth result is " << z << '\n';
```

```
  return 0;
}
```

Output would be:
The first result is 5
The second result is 5
The third result is 2
The fourth result is 6

In this case we have created a function called subtraction. The only thing that this function does is to subtract both passed parameters and to return the result.

Nevertheless, if we examine function main we will see that we have made several calls to function subtraction. We have used some different calling methods so that you see other ways or moments when a function can be called.

In order to understand well these examples you must consider once again that a call to a function could be replaced by the value that the function call itself is going to return. For example, the first case (that you should already know because it is the same pattern that we have used in previous examples):

```
Z = subtraction (7,2);
cout << "The first result is " << z;
```

If we replace the function call by the it returns (i.e., 5), we would have:

```
Z = 5;
cout << "The first result is " << z;
```

As well as

```
cout << "The second result is " << subtraction (7,2);
```

has the same result as the previous call, but in this case we made the call to subtraction directly as an insertion parameter for cout. Simply consider that the result is the same as if we had written:

```
cout << "The second result is " << 5;
```

since 5 is the value returned by subtraction (7,2).

In the case of:

```
cout << "The third result is " << subtraction (x,y);
```

The only new thing that we introduced is that the parameters of subtraction are variables instead of constants. That is perfectly valid. In this case the values passed to function subtraction are the values of x and y, that are 5 and 3 respectively, giving 2 as result.

The fourth case is more of the same. Simply note that instead of:

```
z = 4 + subtraction (x,y);
```

we could have written:

```
z = subtraction (x,y) + 4;
```

with exactly the same result. we have switched places so you can see that the semicolon sign (;) goes at the end of the whole statement. It does not necessarily have to go right after the function call. The explanation might be once again that you imagine that a function can be replaced by its returned value:

```
z = 4 + 2;
z = 2 + 4;
```

Functions with no type. The use of void.

If you remember the syntax of a function declaration:

```
type name ( argument1, argument2 ...) statement
```

you will see that the declaration begins with a type, that is the type of the function itself (i.e., the type of the datum that will be returned by the function with the return statement). But what if we want to return no value?

Imagine that we want to make a function just to show a message on the screen. We do not need it to return any value. In this case we should use the void type specifier for the function. This is a special specifier that indicates absence of type.

```
// void function example
#include <iostream>
using namespace std;

void printmessage ()
{
  cout << "I'm a function!";
}

int main ()
```

```
{
 printmessage ();
 return 0;
}
```

Output would be:
I'm a function!

void can also be used in the function's parameter list to explicitly specify that we want the function to take no actual parameters when it is called. For example, function printmessage could have been declared as:

```
void printmessage (void)
{
 cout << "I'm a function!";
}
```

Although it is optional to specify void in the parameter list. In C++, a parameter list can simply be left blank if we want a function with no parameters.

What you must always remember is that the format for calling a function includes specifying its name and enclosing its parameters between parentheses. The non-existence of parameters does not exempt us from the obligation to write the parentheses. For that reason the call to printmessage is:

```
printmessage ();
```

The parentheses clearly indicate that this is a call to a function and not the name of a variable or some other C++ statement. The following call would have been incorrect:

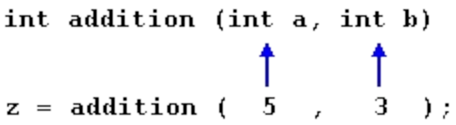
```
printmessage;2
```

Arguments passed by value and by reference.

Until now, in all the functions we have seen, the arguments passed to the functions have been passed by value. This means that when calling a function with parameters, what we have passed to the function were copies of their values but never the variables themselves. For example, suppose that we called our first function addition using the following code:

```
int x=5, y=3, z;
z = addition ( x , y );
```

What we did in this case was to call to function addition passing the values of x and y, i.e. 5 and 3 respectively, but not the variables x and y themselves.



This way, when the function addition is called, the value of its local variables a and b become 5 and 3 respectively, but any modification to either a or b within the function addition will not have any effect in the values of x and y outside it, because variables x and y were not themselves passed to the the function, but only copies of their values at the moment the function was called.

But there might be some cases where you need to manipulate from inside a function the value of an external variable. For that purpose we can use arguments passed by reference, as in the function duplicate of the following example:

```
// passing parameters by reference
#include <iostream>
using namespace std;

void duplicate (int& a, int& b, int& c)
{
  a*=2;
  b*=2;
  c*=2;
}

int main ()
{
  int x=1, y=3, z=7;
  duplicate (x, y, z);
  cout << "x=" << x << ", y=" << y << ", z=" << z;
  return 0;
}
```

output would be:
x=2, y=6, z=14

The first thing that should call your attention is that in the declaration of duplicate the type of each parameter was followed by an ampersand sign (&). This ampersand is what specifies that their corresponding arguments are to be passed by reference instead of by value.

When a variable is passed by reference we are not passing a copy of its value, but we are somehow passing the variable itself to the function and any modification that we do to the local variables will have an effect in their counterpart variables passed as arguments in the call to the function.

To explain it in another way, we associate a, b and c with the arguments passed on the function call (x, y and z) and any change that we do on a within the function will affect the value of x outside it. Any change that we do on b will affect y, and the same with c and z.

That is why our program's output, that shows the values stored in x, y and z after the call to duplicate, shows the values of all the three variables of main doubled.

If when declaring the following function:

```
void duplicate (int& a, int& b, int& c)
```

we had declared it this way:

```
void duplicate (int a, int b, int c)
```

i.e., without the ampersand signs (&), we would have not passed the variables by reference, but a copy of their values instead, and therefore, the output on screen of our program would have been the values of x, y and z without having been modified.

Passing by reference is also an effective way to allow a function to return more than one value. For example, here is a function that returns the previous and next numbers of the first parameter passed.

```
// more than one returning value
#include <iostream>
using namespace std;

void prevnext (int x, int& prev, int& next)
{
  prev = x-1;
  next = x+1;
}

int main ()
{
  int x=100, y, z;
  prevnext (x, y, z);
  cout << "Previous=" << y << ", Next=" << z;
  return 0;
}
```

Output would be:
Previous=99, Next=101

Default values in parameters.

When declaring a function we can specify a default value for each parameter. This value will be used if the corresponding argument is left blank when calling to the function. To do that, we simply have to use the assignation operator and a value for the arguments in the function declaration. If a value for that parameter is not passed when the function is called, the default value is used, but if a value is specified this default value is ignored and the passed value is used instead. For example:

```
// default values in functions
#include <iostream>
using namespace std;

int divide (int a, int b=2)
{
  int r;
  r=a/b;
  return (r);
}

int main ()
{
  cout << divide (12);
  cout << endl;
  cout << divide (20,4);
  return 0;
}
```

Output would be:
6
5

As we can see in the body of the program there are two calls to function divide. In the first one:

```
divide (12)
```

we have only specified one argument, but the function divide allows up to two. So the function divide has assumed that the second parameter is 2 since that is what we have specified to happen if this parameter was not passed (notice the function declaration, which finishes with int b=2, not just int b). Therefore the result of this function call is 6 (12/2).

In the second call:

```
divide (20,4)
```

there are two parameters, so the default value for b (int b=2) is ignored and b takes the value passed as argument, that is 4, making the result returned equal to 5 (20/4).

Overloaded functions.

In C++ two different functions can have the same name if their parameter types or number are different. That means that you can give the same name to more than one function if they have either a different number of parameters or different types in their parameters. For example:

```
// overloaded function
#include <iostream>
using namespace std;

int operate (int a, int b)
{
  return (a*b);
}

float operate (float a, float b)
{
  return (a/b);
}

int main ()
{
  int x=5,y=2;
  float n=5.0,m=2.0;
  cout << operate (x,y);
  cout << "\n";
  cout << operate (n,m);
  cout << "\n";
  return 0;
}
```

output would be:
10
2.5

In this case we have defined two functions with the same name, operate, but one of them accepts two parameters of type int and the other one accepts them of type float. The compiler knows which one to call in each case by examining the types passed as arguments when the function is called. If it is called with two ints as its arguments it calls to the function that has two int parameters in its prototype and if it is called with two floats it will call to the one which has two float parameters in its prototype.

In the first call to operate the two arguments passed are of type int, therefore, the function with the first prototype is called; This function returns the result of multiplying both parameters. While the second call passes two arguments of type float, so the function with the second prototype is called. This one has a different bahavior: it divides one parameter by the other. So the behavior of a call to operate depends on the type of the arguments passed because the function has been overloaded.

Please notice that a function cannot be overloaded only by its return type. At least one of its parameters must have a different type.

inline functions.

The inline specifier is an optimization specifier. It does not change the behavior of a function, but serves to indicate the compiler that the code the function body generates shall be inserted at the point of each call to the function, instead of doing a regular call that generally involves stacking variables and jumping to other memory locations. This is equivalent to declaring a macro. It only represents an overhead advantage for very short functions, in which the resulting code from compiling the program may be faster when the overhead required to call a function is avoided.

The format for its declaration is:

inline type name (arguments ...) { instructions ... }

and the call is just like the call to any other function. You do not have to include the inline keyword when calling the function, only in its declaration.

Most compilers already optimize code to generate inline functions when it is more convenient. This specifier only indicates the compiler that inline is preferred for this function.

Recursivity.

Recursivity is the property that functions have to be called by themselves. It is useful for many tasks, like sorting or calculate the factorial of numbers. For example, to obtain the factorial of a number (n!) the mathematical formula would be:

n! = n * (n-1) * (n-2) * (n-3) ... * 1

more concretely, 5! (factorial of 5) would be:

5! = 5 * 4 * 3 * 2 * 1 = 120

and a recursive function to calculate this in C++ could be:

```
// factorial calculator
#include <iostream>
```

```
using namespace std;

long factorial (long a)
{
  if (a > 1)
   return (a * factorial (a-1));
  else
   return (1);
}

int main ()
{
  long number;
  cout << "Please type a number: ";
  cin >> number;
  cout << number << "! = " << factorial (number);
  return 0;
}
```

Output would be:
Please type a number: 9
9! = 362880

Notice how in function factorial we included a call to itself, but only if the argument passed was greater than 1, since otherwise the function would perform an infinite recursive loop in which once it arrived to 0 it would continue multiplying by all the negative numbers (probably provoking a stack overflow error on runtime).

This function has a limitation because of the data type we used in its design (long) for more simplicity. The results given will not be valid for values much greater than 10! or 15!, depending on the system you compile it.

Declaring functions.

Until now, we have defined all of the functions before the first appearance of calls to them in the source code. These calls were generally in function main which we have always left at the end of the source code. But if you try to repeat some of the examples of functions described so far, but placing the function main before any of the other functions that were called from within it, you will most likely obtain compiling errors. The reason is that to be able to call a function this must have been declared in some earlier point of the code, like we have done in all our examples.

But there is an alternative way to avoid writing the whole code of a function before it can be used in main or in some other function. This can be achieved by declaring just a prototype of the function before it is used, instead of the entire definition. This declaration is shorter than the entire definition, but enough significant for the compiler to determine its return type and the types of its parameters.

Its form is:

type name (argument_type1, argument_type2, ...);

It is identical to a function definition, except that it does not include the body of the function itself (i.e., the function statements that in normal definitions are enclosed in braces { }) and instead of that we end the prototype declaration with a mandatory semicolon (;).

The parameter enumeration does not need to include the identifiers, but only the type specifiers. The inclusion of a name for each parameter as in the function definition is optional in the prototype declaration. For example, we can declare a function called protofunction with two int parameters with any of the following declarations:

```
int protofunction (int first, int second);
int protofunction (int, int);
```

Anyway, including a name for each variable makes the prototype more legible.

```
// declaring functions prototypes
#include <iostream>
using namespace std;

void odd (int a);
void even (int a);

int main ()
{
  int i;
  do {
    cout << "Type a number: (0 to exit) ";
    cin >> i;
    odd (i);
  } while (i!=0);
  return 0;
}

void odd (int a)
{
  if ((a%2)!=0) cout << "Number is odd.\n";
  else even (a);
}

void even (int a)
{
  if ((a%2)==0) cout << "Number is even.\n";
  else odd (a);
}
```

Output would be:
Type a number (0 to exit): 9
Number is odd.
Type a number (0 to exit): 6
Number is even.
Type a number (0 to exit): 1030
Number is even.
Type a number (0 to exit): 0
Number is even.

This example is indeed not an example of efficiency. we are sure that at this point you can already make a program with the same result, but using only half of the code lines that have been used in this example. Anyway this example illustrates how prototyping works. Moreover, in this concrete example the prototyping of at least one of the two functions is necessary in order to compile the code without errors.

The first things that we see are the declaration of functions odd and even:

```
void odd (int a);
void even (int a);
```

This allows these functions to be used before they are defined, for example, in main, which now is located where some people find it to be a more logical place for the start of a program: the beginning of the source code.

Anyway, the reason why this program needs at least one of the functions to be declared before it is defined is because in odd there is a call to even and in even there is a call to odd. If none of the two functions had been previously declared, a compilarion error would happen, since either odd would not not be visible from even (because it has still not been declared), or even would not be visible from odd (for the same reason).

Having the prototype of all functions together in the same place within the source code is found practical by some programmers, and this can be easily achieved by declaring all functions prototypes at the beginning of a program.

Pointer initialization

When declaring pointers we may want to explicitly specify which variable we want them to point to:

```
int number;
int *tommy = &number;
```

The behavior of this code is equivalent to:

```
int number;
int *tommy;
tommy = &number;
```

When a pointer initialization takes place we are always assigning the reference value to where the pointer points (tommy), never the value being pointed (*tommy). You must consider that at the moment of declaring a pointer, the asterisk (*) indicates only that it is a pointer, it is not the reference operator (although both use the same sign: *). Remember, they are two different functions of one sign. Thus, we must take care not to confuse the previous code with:

```
int number;
int *tommy;
*tommy = &number;
```

that is incorrect, and anyway would not have much sense in this case if you think about it.

As in the case of arrays, the compiler allows the special case that we want to initialize the content at which the pointer points with constants at the same moment the pointer is declared:

```
char * terry = "hello";
```

In this case, memory space is reserved to contain "hello" and then a pointer to the first character of this memory block is assigned to terry. If we imagine that "hello" is stored at the memory locations that start at addresses 1702, we can represent the previous declaration as:

It is important to indicate that terry contains the value 1702, and not 'h' nor "hello", although 1702 indeed is the address of both of these.

The pointer terry points to a sequence of characters and can be read as if it was an Array(remember that an array is just like a constant pointer). For example, we can access the fifth element of the array with any of these two expression:

```
*(terry+4)
terry[4]
```

Both expressions have a value of 'o' (the fifth element of the array).

Pointer arithmetics

To conduct arithmetical operations on pointers is a little different than to conduct them on regular integer data types. To begin with, only addition and subtraction operations are allowed to be conducted with them, the others make no sense in the world of pointers. But both addition and subtraction have a different behavior with pointers according to the size of the data type to which they point.

When we saw the different fundamental data types, we saw that some occupy more or less space than others in the memory. For example, in the case of integer numbers, char takes 1 byte, short takes 2 bytes and long takes 4.

Suppose that we have three pointers:

```
char *mychar;
short *myshort;
long *mylong;
```

and that we know that they point to memory locations 1000, 2000 and 3000 respectively.

So if we write:

```
mychar++;
myshort++;
mylong++;
```

mychar, as you may expect, would contain the value 1001. Nevertheless, myshort would contain the value 2002, and mylong would contain 3004. The reason is that when adding one to a pointer we are making it to point to the following element of the same type with which it has been defined, and therefore the size in bytes of the type pointed is added to the pointer.

This is applicable both when adding and subtracting any number to a pointer. It would happen exactly the same if we write:

```
mychar = mychar + 1;
myshort = myshort + 1;
mylong = mylong + 1;
```

Both the increase (++) and decrease (--) operators have greater operator precedence than the dereference operator (*), but both have a special behavior when used as suffix (the expression is evaluated with the value it had before being increased). Therefore, the following expression may lead to confusion:

```
*p++
```

Because ++ has greater precedence than *, this expression is equivalent to *(p++). Therefore, what it does is to increase the value of p (so it now points to the next element), but because ++ is used as postfix the whole expression is evaluated as the value pointed by the original reference (the address the pointer pointed to before being increased).

Notice the difference with:

```
(*p)++
```

Here, the expression would have been evaluated as the value pointed by p increased by one. The value of p (the pointer itself) would not be modified (what is being modified is what it is being pointed to by this pointer).

If we write:

```
*p++ = *q++;
```

Because ++ has a higher precedence than *, both p and q are increased, but because both increase operators (++) are used as postfix and not prefix, the value assigned to *p is *q before both p and q are increased. And then both are increased. It would be roughly equivalent to:

```
*p = *q;
++p;
++q;
```

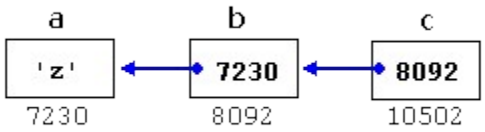
Like always, I recommend you to use parentheses () in order to avoid unexpected results and to give more legibility.

Pointers to pointers

C++ allows the use of pointers that point to pointers, that these, in its turn, point to data (or even to other pointers). In order to do that, we only need to add an asterisk (*) for each level of reference in their declarations:

```
char a;
char * b;
char ** c;
a = 'z';
b = &a;
c = &b;
```

This, supposing the randomly chosen memory locations for each variable of 7230, 8092 and 10502, could be represented as:



The value of each variable is written inside each cell; under the cells are their respective addresses in memory.

The new thing in this example is variable c, which can be used in three different levels of indirection, each one of them would correspond to a different value:

- c has type char** and a value of 8092
- *c has type char* and a value of 7230
- **c has type char and a value of 'z'

void pointers

The void type of pointer is a special type of pointer. In C++, void represents the absence of type, so void pointers are pointers that point to a value that has no type (and thus also an undetermined length and undetermined dereference properties). This allows void pointers to point to any data type, from an integer value or a float to a string of characters. But in exchange they have a great limitation: the data pointed by them cannot be directly dereferenced (which is logical, since we have no type to dereference to), and for that reason we will always have to change the type of the void pointer to some other pointer type that points to a concrete data type before dereferencing it. This is done by performing type-castings.

One of its uses may be to pass generic parameters to a function:

```
// increaser
#include <iostream>
using namespace std;

void increase (void* data, int size)
{
  switch (size)
  {
    case sizeof(char) : (*((char*)data))++; break;
    case sizeof(int) : (*((int*)data))++; break;
  }
}

int main ()
{
```

```
  char a = 'x';
  int b = 1602;
  increase (&a,sizeof(a));
  increase (&b,sizeof(b));
  cout << a << ", " << b << endl;
  return 0;
}
```

Output would be:
y, 1603

sizeof is an operator integrated in the C++ language that returns the size in bytes of its parameter. For non-dynamic data types this value is a constant. Therefore, for example, sizeof(char) is 1, because char type is one byte long.

Null pointer

A null pointer is a regular pointer of any pointer type which has a special value that indicates that it is not pointing to any valid reference or memory address. This value is the result of type-casting the integer value zero to any pointer type.

```
int * p;
p = 0;    // p has a null pointer value
```

Do not confuse null pointers with void pointers. A null pointer is a value that any pointer may take to represent that it is pointing to "nowhere", while a void pointer is a special type of pointer that can point to somewhere without a specific type. One refers to the value stored in the pointer itself and the other to the type of data it points to.

Pointers to functions

C++ allows operations with pointers to functions. The typical use of this is for passing a function as an argument to another function, since these cannot be passed dereferenced. In order to declare a pointer to a function we have to declare it like the prototype of the function except that the name of the function is enclosed between parentheses () and an asterisk (*) is inserted before the name:

```
// pointer to functions
#include <iostream>
using namespace std;

int addition (int a, int b)
{ return (a+b); }

int subtraction (int a, int b)
{ return (a-b); }
```

```
int (*minus)(int,int) = subtraction;

int operation (int x, int y, int (*functocall)(int,int))
{
  int g;
  g = (*functocall)(x,y);
  return (g);
}

int main ()
{
  int m,n;
  m = operation (7, 5, addition);
  n = operation (20, m, minus);
  cout <<n;
  return 0;
}
```

Output would be:
8

In the example, minus is a global pointer to a function that has two parameters of type int. It is immediately assigned to point to the function subtraction, all in a single line:

```
int (* minus)(int,int) = subtraction;
```

Dynamic memory

Until now, in all our programs, we have only had as much memory available as we declared for our variables, having the size of all of them to be determined in the source code, before the execution of the program. But, what if we need a variable amount of memory that can only be determined during runtime? For example, in the case that we need some user input to determine the necessary amount of memory space.

The answer is dynamic memory, for which C++ integrates the operators new and delete.

Operators new and new[]

In order to request dynamic memory we use the operator new. new is followed by a data type specifier and -if a sequence of more than one element is required- the number of these within brackets []. It returns a pointer to the beginning of the new block of memory allocated. Its form is:

pointer = new type
pointer = new type [elements]

The first expression is used to allocate memory to contain one single element of type type. The second one is used to assign a block (an array) of elements of type type, where elements is an integer value representing the number of these. For example:

```
int * bobby;
bobby = new int [5];
```

In this case, the system dynamically assigns space for five elements of type int and returns a pointer to the first element of the sequence, which is assigned to bobby. Therefore, now, bobby points to a valid block of memory with space for five elements of type int.

The first element pointed by bobby can be accessed either with the expression bobby[0] or the expression *bobby. Both are equivalent as has been explained in the section about pointers. The second element can be accessed either with bobby[1] or *(bobby+1) and so on...

You could be wondering the difference between declaring a normal array and assigning dynamic memory to a pointer, as we have just done. The most important difference is that the size of an array has to be a constant value, which limits its size to what we decide at the moment of designing the program, before its execution, whereas the dynamic memory allocation allows us to assign memory during the execution of the program (runtime) using any variable or constant value as its size.

The dynamic memory requested by our program is allocated by the system from the memory heap. However, computer memory is a limited resource, and it can be exhausted. Therefore, it is important to have some mechanism to check if our request to allocate memory was successful or not.

C++ provides two standard methods to check if the allocation was successful:

One is by handling exceptions. Using this method an exception of type bad_alloc is thrown when the allocation fails. Exceptions are a powerful C++ feature explained later in these tutorials. But for now you should know that if this exception is thrown and it is not handled by a specific handler, the program execution is terminated.

This exception method is the default method used by new, and is the one used in a declaration like:

```
bobby = new int [5];  // if it fails an exception is thrown
```

The other method is known as nothrow, and what happens when it is used is that when a memory allocation fails, instead of throwing a bad_alloc exception or terminating the program, the pointer returned by new is a null pointer, and the program continues its execution.

This method can be specified by using a special object called nothrow as parameter for new:

```
bobby = new (nothrow) int [5];
```

In this case, if the allocation of this block of memory failed, the failure could be detected by checking if bobby took a null pointer value:

```
int * bobby;
bobby = new (nothrow) int [5];
if (bobby == 0) {
  // error assigning memory. Take measures.
  };
```

This nothrow method requires more work than the exception method, since the value returned has to be checked after each and every memory allocation, but we will use it in our examples due to its simplicity. Anyway this method can become tedious for larger projects, where the exception method is generally preferred. The exception method will be explained in detail later in this tutorial.

Operator delete and delete[]

Since the necessity of dynamic memory is usually limited to specific moments within a program, once it is no longer needed it should be freed so that the memory becomes available again for other requests of dynamic memory. This is the purpose of the operator delete, whose format is:

```
delete pointer;
delete [] pointer;
```

The first expression should be used to delete memory allocated for a single element, and the second one for memory allocated for arrays of elements.

The value passed as argument to delete must be either a pointer to a memory block previously allocated with new, or a null pointer (in the case of a null pointer, delete produces no effect).

```
// rememb-o-matic
#include <iostream>
using namespace std;

int main ()
{
  int i,n;
  int * p;
  cout << "How many numbers would you like to type? ";
  cin >> i;
  p= new (nothrow) int[i];
  if (p == 0)
    cout << "Error: memory could not be allocated";
  else
  {
    for (n=0; n<i; n++)
    {
      cout << "Enter number: ";
      cin >> p[n];
    }
    cout << "You have entered: ";
    for (n=0; n<i; n++)
      cout << p[n] << ", ";
    delete[] p;
  }
  return 0;
}
```

Output would be:
How many numbers would you like to type? 5
Enter number: 75
Enter number: 436
Enter number: 1067
Enter number: 8
Enter number: 32
You have entered: 75, 436, 1067, 8, 32,

Notice how the value within brackets in the new statement is a variable value entered by the user (i), not a constant value:

```
P= new (nothrow) int[i];
```

But the user could have entered a value for i so big that our system could not handle it. For example, when I tried to give a value of 1 billion to the "How many numbers" question, my system could not allocate that much memory for the program and I got the text message we prepared for this case (Error: memory could not be allocated). Remember that in the case that we tried to allocate the memory without

specifying the nothrow parameter in the new expression, an exception would be thrown, which if it's not handled terminates the program.

It is a good practice to always check if a dynamic memory block was successfully allocated. Therefore, if you use the nothrow method, you should always check the value of the pointer returned. Otherwise, use the exception method, even if you do not handle the exception. This way, the program will terminate at that point without causing the unexpected results of continuing executing a code that assumes a block of memory to have been allocated when in fact it has not.

Data structures

We have already learned how groups of sequential data can be used in C++. But this is somewhat restrictive, since in many occasions what we want to store are not mere sequences of elements all of the same data type, but sets of different elements with different data types.

A data structure is a group of data elements grouped together under one name. These data elements, known as members, can have different types and different lengths. Data structures are declared in C++ using the following syntax:

```
struct structure_name {
member_type1 member_name1;
member_type2 member_name2;
member_type3 member_name3;
.
.
} object_names;
```

where structure_name is a name for the structure type, object_name can be a set of valid identifiers for objects that have the type of this structure. Within braces { } there is a list with the data members, each one is specified with a type and a valid identifier as its name.

The first thing we have to know is that a data structure creates a new type: Once a data structure is declared, a new type with the identifier specified as structure_name is created and can be used in the rest of the program as if it was any other type. For example:

```
struct product {
  int weight;
  float price;
} ;

product apple;
product banana, melon;
```

We have first declared a structure type called product with two members: weight and price, each of a different fundamental type. We have then used this name of the structure type (product) to declare three objects of that type: apple, banana and melon as we would have done with any fundamental data type.

Once declared, product has become a new valid type name like the fundamental ones int, char or short and from that point on we are able to declare objects (variables) of this compound new type, like we have done with apple, banana and melon.

Right at the end of the struct declaration, and before the ending semicolon, we can use the optional field object_name to directly declare objects of the structure type. For example, we can also declare the structure objects apple, banana and melon at the moment we define the data structure type this way:

```
struct product {
  int weight;
  float price;
} apple, banana, melon;
```

It is important to clearly differentiate between what is the structure type name, and what is an object (variable) that has this structure type. We can instantiate many objects (i.e. variables, like apple, banana and melon) from a single structure type (product).

Once we have declared our three objects of a determined structure type (apple, banana and melon) we can operate directly with their members. To do that we use a dot (.) inserted between the object name and the member name. For example, we could operate with any of these elements as if they were standard variables of their respective types:

```
apple.weight
apple.price
banana.weight
banana.price
melon.weight
melon.price
```

Each one of these has the data type corresponding to the member they refer to: apple.weight, banana.weight and melon.weight are of type int, while apple.price, banana.price and melon.price are of type float.

Let's see a real example where you can see how a structure type can be used in the same way as fundamental types:

```
// example about structures
#include <iostream>
#include <string>
#include <sstream>
using namespace std;

struct movies_t {
  string title;
  int year;
} mine, yours;

void printmovie (movies_t movie);

int main ()
{
  string mystr;

  mine.title = "2001 A Space Odyssey";
  mine.year = 1968;

  cout << "Enter title: ";
  getline (cin,yours.title);
  cout << "Enter year: ";
  getline (cin,mystr);
  stringstream(mystr) >> yours.year;

  cout << "My favorite movie is:\n ";
  printmovie (mine);
  cout << "And yours is:\n ";
  printmovie (yours);
  return 0;
}

void printmovie (movies_t movie)
{
  cout << movie.title;
  cout << " (" << movie.year << ")\n";
}
```

Output would be:
Enter title: Alien

```
Enter year: 1979

My favorite movie is:
 2001 A Space Odyssey (1968)
And yours is:
 Alien (1979)
```

The example shows how we can use the members of an object as regular variables. For example, the member yours.year is a valid variable of type int, and mine.title is a valid variable of type string.

The objects mine and yours can also be treated as valid variables of type movies_t, for example we have passed them to the function printmovie as we would have done with regular variables. Therefore, one of the most important advantages of data structures is that we can either refer to their members individually or to the entire structure as a block with only one identifier.

Data structures are a feature that can be used to represent databases, especially if we consider the possibility of building arrays of them:

```
// array of structures
#include <iostream>
#include <string>
#include <sstream>
using namespace std;

#define N_MOVIES 3

struct movies_t {
  string title;
  int year;
} films [N_MOVIES];

void printmovie (movies_t movie);

int main ()
{
  string mystr;
  int n;

  for (n=0; n<N_MOVIES; n++)
  {
    cout << "Enter title: ";
    getline (cin,films[n].title);
    cout << "Enter year: ";
    getline (cin,mystr);
    stringstream(mystr) >> films[n].year;
  }
```

```
  cout << "\nYou have entered these movies:\n";
  for (n=0; n<N_MOVIES; n++)
    printmovie (films[n]);
  return 0;
}

void printmovie (movies_t movie)
{
  cout << movie.title;
  cout << " (" << movie.year << ")\n";
}
```

Output would be:

```
Enter title: Blade Runner
Enter year: 1982
Enter title: Matrix
Enter year: 1999
Enter title: Taxi Driver
Enter year: 1976

You have entered these movies:
Blade Runner (1982)
Matrix (1999)
Taxi Driver (1976

)
```

Pointers to structures

Like any other type, structures can be pointed by its own type of pointers:

```
struct movies_t {
  string title;
  int year;
};

movies_t amovie;
movies_t * pmovie;
```

Here amovie is an object of structure type movies_t, and pmovie is a pointer to point to objects of structure type movies_t. So, the following code would also be valid:

```
pmovie = &amovie;
```

The value of the pointer pmovie would be assigned to a reference to the object amovie (its memory address).

We will now go with another example that includes pointers, which will serve to introduce a new operator: the arrow operator (->):

```
// pointers to structures
#include <iostream>
#include <string>
#include <sstream>
using namespace std;

struct movies_t {
  string title;
  int year;
};

int main ()
{
  string mystr;

  movies_t amovie;
  movies_t * pmovie;
  pmovie = &amovie;

  cout << "Enter title: ";
  getline (cin, pmovie->title);
  cout << "Enter year: ";
  getline (cin, mystr);
  (stringstream) mystr >> pmovie->year;

  cout << "\nYou have entered:\n";
  cout << pmovie->title;
  cout << " (" << pmovie->year << ")\n";

  return 0;
}
```

Output would be:

```
Enter title: Invasion of the body snatchers
Enter year: 1978

You have entered:
Invasion of the body snatchers (1978)
```

The previous code includes an important introduction: the arrow operator (->). This is a dereference operator that is used exclusively with pointers to objects with

members. This operator serves to access a member of an object to which we have a reference. In the example we used:

```
pmovie->title
```

Which is for all purposes equivalent to:

```
(*pmovie).title
```

Both expressions pmovie->title and (*pmovie).title are valid and both mean that we are evaluating the member title of the data structure pointed by a pointer called pmovie. It must be clearly differentiated from:

```
*pmovie.title
```

which is equivalent to:

```
*(pmovie.title)
```

And that would access the value pointed by a hypothetical pointer member called title. The following panel summarizes possible combinations of pointers and structure members:

Expression	What is evaluated	Equivalent
a.b	Member b of object a	
a->b	Member b of object pointed by a	(*a).b
*a.b	Value pointed by member b of object a	*(a.b)

Nesting structures

Structures can also be nested so that a valid element of a structure can also be on its turn another structure.

```
struct movies_t {
  string title;
  int year;
};

struct friends_t {
  string name;
  string email;
  movies_t favorite_movie;
```

```
} charlie, maria;

friends_t * pfriends = &charlie;
```

After the previous declaration we could use any of the following expressions:

```
charlie.name
maria.favorite_movie.title
charlie.favorite_movie.year
pfriends->favorite_movie.year
```

(where, by the way, the last two expressions refer to the same member).

Other Data Types

Defined data types (typedef)

C++ allows the definition of our own types based on other existing data types. We can do this using the keyword typedef, whose format is:

typedef existing_type new_type_name ;

where existing_type is a C++ fundamental or compound type and new_type_name is the name for the new type we are defining. For example:

```
typedef char C;
typedef unsigned int WORD;
typedef char * pChar;
typedef char field [50];
```

In this case we have defined four data types: C, WORD, pChar and field as char, unsigned int, char* and char[50] respectively, that we could perfectly use in declarations later as any other valid type:

```
C mychar, anotherchar, *ptc1;
WORD myword;
pChar ptc2;
field name;
```

typedef does not create different types. It only creates synonyms of existing types. That means that the type of myword can be considered to be either WORD or unsigned int, since both are in fact the same type.

typedef can be useful to define an alias for a type that is frequently used within a program. It is also useful to define types when it is possible that we will need to

change the type in later versions of our program, or if a type you want to use has a name that is too long or confusing.

Classes

A class is an expanded concept of a data structure: instead of holding only data, it can hold both data and functions.

An object is an instantiation of a class. In terms of variables, a class would be the type, and an object would be the variable.

Classes are generally declared using the keyword class, with the following format:

```
class class_name {
  access_specifier_1:
    member1;
  access_specifier_2:
    member2;
  ...
  } object_names;
```

Where class_name is a valid identifier for the class, object_names is an optional list of names for objects of this class. The body of the declaration can contain members, that can be either data or function declarations, and optionally access specifiers.

All is very similar to the declaration on data structures, except that we can now include also functions and members, but also this new thing called access specifier. An access specifier is one of the following three keywords: private, public or protected. These specifiers modify the access rights that the members following them acquire:

- private members of a class are accessible only from within other members of the same class or from their *friends*.
- protected members are accessible from members of their same class and from their friends, but also from members of their derived classes.
- Finally, public members are accessible from anywhere where the object is visible.

By default, all members of a class declared with the class keyword have private access for all its members. Therefore, any member that is declared before one other class specifier automatically has private access. For example:

```
class CRectangle {
    int x, y;
  public:
    void set_values (int,int);
    int area (void);
  } rect;
```

Declares a class (i.e., a type) called CRectangle and an object (i.e., a variable) of this class called rect. This class contains four members: two data members of type int (member x and member y) with private access (because private is the default access level) and two member functions with public access: set_values() and area(), of which for now we have only included their declaration, not their definition.

Notice the difference between the class name and the object name: In the previous example, CRectangle was the class name (i.e., the type), whereas rect was an object of type CRectangle. It is the same relationship int and a have in the following declaration:

```
int a;
```

where int is the type name (the class) and a is the variable name (the object).

After the previous declarations of CRectangle and rect, we can refer within the body of the program to any of the public members of the object rect as if they were normal functions or normal variables, just by putting the object's name followed by a dot (.) and then the name of the member. All very similar to what we did with plain data structures before. For example:

```
rect.set_values (3,4);
myarea = rect.area();
```

The only members of rect that we cannot access from the body of our program outside the class are x and y, since they have private access and they can only be referred from within other members of that same class.

Here is the complete example of class CRectangle:

```
// classes example
#include <iostream>
using namespace std;

class CRectangle {
    int x, y;
  public:
    void set_values (int,int);
    int area () {return (x*y);}
};
```

```
void CRectangle::set_values (int a, int b) {
  x = a;
  y = b;
}

int main () {
  CRectangle rect;
  rect.set_values (3,4);
  cout << "area: " << rect.area();
  return 0;
}
```

Output would be:

```
area: 12
```

The most important new thing in this code is the operator of scope (::, two colons) included in the definition of set_values(). It is used to define a member of a class from outside the class declaration itself.

You may notice that the definition of the member function area() has been included directly within the definition of the CRectangle class given its extreme simplicity, whereas set_values() has only its prototype declared within the class, but its definition is outside it. In this outside declaration, we must use the operator of scope (::) to specify that we are defining a function that is a member of the class CRectangle and not a regular global function.

The scope operator (::) specifies the class to which the member being declared belongs, granting exactly the same scope properties as if this function definition was directly included within the class definition. For example, in the function set_values() of the previous code, we have been able to use the variables x and y, which are private members of class CRectangle, which means they are only accessible from other members of their class.

The only difference between defining a class member function completely within its class and to include only the prototype and later its definition, is that in the first case the function will automatically be considered an inline member function by the compiler, while in the second it will be a normal (not-inline) class member function, which in fact supposes no difference in behavior.

Members x and y have private access (remember that if nothing else is said, all members of a class defined with keyword class have private access). By declaring them private we deny access to them from anywhere outside the class. This makes sense, since we have already defined a member function to set values for those members within the object: the member function set_values(). Therefore, the rest of the program does not need to have direct access to them. Perhaps in a so simple example as this, it is difficult to see an utility in protecting those two variables, but in greater projects it may be very important that values cannot be modified in an unexpected way (unexpected from the point of view of the object).

One of the greater advantages of a class is that, as any other type, we can declare several objects of it. For example, following with the previous example of class CRectangle, we could have declared the object rectb in addition to the object rect:

```
// example: one class, two objects
#include <iostream>
using namespace std;

class CRectangle {
    int x, y;
  public:
    void set_values (int,int);
    int area () {return (x*y);}
};

void CRectangle::set_values (int a, int b) {
  x = a;
  y = b;
}

int main () {
  CRectangle rect, rectb;
  rect.set_values (3,4);
  rectb.set_values (5,6);
  cout << "rect area: " << rect.area() << endl;
  cout << "rectb area: " << rectb.area() << endl;
  return 0;
}
```

Output would be:

```
rect area: 12
rectb area: 30
```

In this concrete case, the class (type of the objects) to which we are talking about is CRectangle, of which there are two instances or objects: rect and rectb. Each one of them has its own member variables and member functions.

Notice that the call to rect.area() does not give the same result as the call to rectb.area(). This is because each object of class CRectangle has its own variables x and y, as they, in some way, have also their own function members set_value() and area() that each uses its object's own variables to operate.

That is the basic concept of object-oriented programming: Data and functions are both members of the object. We no longer use sets of global variables that we pass from one function to another as parameters, but instead we handle objects that have their own data and functions embedded as members. Notice that we have not had to

give any parameters in any of the calls to rect.area or rectb.area. Those member functions directly used the data members of their respective objects rect and rectb.

Constructors and destructors

Objects generally need to initialize variables or assign dynamic memory during their process of creation to become operative and to avoid returning unexpected values during their execution. For example, what would happen if in the previous example we called the member function area() before having called function set_values()? Probably we would have gotten an undetermined result since the members x and y would have never been assigned a value.

In order to avoid that, a class can include a special function called constructor, which is automatically called whenever a new object of this class is created. This constructor function must have the same name as the class, and cannot have any return type; not even void.

We are going to implement CRectangle including a constructor:

```
// example: class constructor
#include <iostream>
using namespace std;

class CRectangle {
    int width, height;
  public:
    CRectangle (int,int);
    int area () {return (width*height);}
};

CRectangle::CRectangle (int a, int b) {
  width = a;
  height = b;
}

int main () {
  CRectangle rect (3,4);
  CRectangle rectb (5,6);
  cout << "rect area: " << rect.area() << endl;
  cout << "rectb area: " << rectb.area() << endl;
  return 0;
}
```

Output would be:

```
rect area: 12
rectb area: 30
```

As you can see, the result of this example is identical to the previous one. But now we have removed the member function set_values(), and have included instead a constructor that performs a similar action: it initializes the values of x and y with the parameters that are passed to it.

Notice how these arguments are passed to the constructor at the moment at which the objects of this class are created:

```
CRectangle rect (3,4);
CRectangle rectb (5,6);
```

Constructors cannot be called explicitly as if they were regular member functions. They are only executed when a new object of that class is created.

You can also see how neither the prototype nor the later constructor declaration includes a return value; not even void.

The destructor fulfills the opposite functionality. It is automatically called when an object is destroyed, either because its scope of existence has finished (for example, if it was defined as a local object within a function and the function ends) or because it is an object dynamically assigned and it is released using the operator delete.

The destructor must have the same name as the class, but preceded with a tilde sign (~) and it must also return no value.

The use of destructors is especially suitable when an object assigns dynamic memory during its lifetime and at the moment of being destroyed we want to release the memory that the object was allocated.

```
// example on constructors and destructors
#include <iostream>
using namespace std;

class CRectangle {
    int *width, *height;
  public:
    CRectangle (int,int);
    ~CRectangle ();
    int area () {return (*width * *height);}
};

CRectangle::CRectangle (int a, int b) {
  width = new int;
  height = new int;
  *width = a;
  *height = b;
```

```
}

CRectangle::~CRectangle () {
  delete width;
  delete height;
}

int main () {
  CRectangle rect (3,4), rectb (5,6);
  cout << "rect area: " << rect.area() << endl;
  cout << "rectb area: " << rectb.area() << endl;
  return 0;
}
```

Output would be:

```
rect area: 12
rectb area: 30
```

Overloading Constructors

Like any other function, a constructor can also be overloaded with more than one function that have the same name but different types or number of parameters. Remember that for overloaded functions the compiler will call the one whose parameters match the arguments used in the function call. In the case of constructors, which are automatically called when an object is created, the one executed is the one that matches the arguments passed on the object declaration:

```
// overloading class constructors
#include <iostream>
using namespace std;

class CRectangle {
    int width, height;
  public:
    CRectangle ();
    CRectangle (int,int);
    int area (void) {return (width*height);}
};

CRectangle::CRectangle () {
  width = 5;
  height = 5;
}

CRectangle::CRectangle (int a, int b) {
  width = a;
  height = b;
```

```
}

int main () {
  CRectangle rect (3,4);
  CRectangle rectb;
  cout << "rect area: " << rect.area() << endl;
  cout << "rectb area: " << rectb.area() << endl;
  return 0;
}
```

Output would be:
rect area: 12
rectb area: 25

In this case, rectb was declared without any arguments, so it has been initialized with the constructor that has no parameters, which initializes both width and height with a value of 5.

Important: Notice how if we declare a new object and we want to use its default constructor (the one without parameters), we do not include parentheses ():

```
CRectangle rectb;   // right
CRectangle rectb(); // wrong!
```

Default constructor

If you do not declare any constructors in a class definition, the compiler assumes the class to have a default constructor with no arguments. Therefore, after declaring a class like this one:

```
class CExample {
  public:
    int a,b,c;
    void multiply (int n, int m) { a=n; b=m; c=a*b; };
  };
```

The compiler assumes that CExample has a default constructor, so you can declare objects of this class by simply declaring them without any arguments:

```
CExample ex;
```

But as soon as you declare your own objects of that class according to the constructor prototypes you defined for the class: constructor for a class, the compiler no longer provides an implicit default constructor. So u have to declare all objects of that class according to constructor prototypes you defined for that class:

```
CExample::CExample (const CExample& rv) {
 a=rv.a;  b=rv.b;  c=rv.c;
 }
  CExample (int n, int m) { a=n; b=m; };
  void multiply () { c=a*b; };
};
```

Here we have declared a constructor that takes two parameters of type int. Therefore the following object declaration would be correct:

```
CExample ex (2,3);
```

But,

```
CExample ex;
```

Would not be correct, since we have declared the class to have an explicit constructor, thus replacing the default constructor.

But the compiler not only creates a default constructor for you if you do not specify your own. It provides three special member functions in total that are implicitly declared if you do not declare your own. These are the copy constructor, the copy assignment operator, and the default destructor.

The copy constructor and the copy assignment operator copy all the data contained in another object to the data members of the current object. For CExample, the copy constructor implicitly declared by the compiler would be something similar to:

Therefore, the two following object declarations would be correct:

```
CExample ex (2,3);
CExample ex2 (ex);   // copy constructor (data copied from ex)
```

Pointers to classes

It is perfectly valid to create pointers that point to classes. We simply have to consider that once declared, a class becomes a valid type, so we can use the class name as the type for the pointer. For example:

```
CRectangle * prect;
```

is a pointer to an object of class CRectangle.

As it happened with data structures, in order to refer directly to a member of an object pointed by a pointer we can use the arrow operator (->) of indirection. Here is an example with some possible combinations:

```
// pointer to classes example
#include <iostream>
using namespace std;

class CRectangle {
    int width, height;
  public:
    void set_values (int, int);
    int area (void) {return (width * height);}
};

void CRectangle::set_values (int a, int b) {
  width = a;
  height = b;
}

int main () {
  CRectangle a, *b, *c;
  CRectangle * d = new CRectangle[2];
  b= new CRectangle;
  c= &a;
  a.set_values (1,2);
  b->set_values (3,4);
  d->set_values (5,6);
  d[1].set_values (7,8);
  cout << "a area: " << a.area() << endl;
  cout << "*b area: " << b->area() << endl;
  cout << "*c area: " << c->area() << endl;
  cout << "d[0] area: " << d[0].area() << endl;
  cout << "d[1] area: " << d[1].area() << endl;
  return 0;
}
```

Output would be:

```
a area: 2
*b area: 12
*c area: 2
d[0] area: 30
d[1] area: 56
```

Next you have a summary on how can you read some pointer and class operators (*, &, ., ->, [ ]) that appear in the previous example:

expression	can be read as

*x	pointed by x
&x	address of x
x.y	member y of object x
x->y	member y of object pointed by x
(*x).y	member y of object pointed by x (equivalent to the previous one)
x[0]	First object pointed by x
x[1]	second object pointed by x
x[n]	(n+1)th object pointed by x

Be sure that you understand the logic under all of these expressions before proceeding with the next sections. If you have doubts, read again this section and/or consult the previous sections about pointers and data structures.

Classes defined with struct and union

Classes can be defined not only with keyword class, but also with keywords struct and union.

The concepts of class and data structure are so similar that both keywords have in C++ the exact same functionality except that the members of classes declared with keyword struct have public access by default, instead of private access, as classes declared with keyword class have. That is the only difference. For all other purposes both keywords are equivalent.

The concept under unions is different from than of struct and class, since unions only store one data member at a time, but they are also classes and thus can also hold function members. The default access in union classes is public.

Classes (II)

Overloading operators

C++ incorporates the option to use standard operators to perform operations with classes in addition to with fundamental types. For example:

```
int a, b, c;
a = b + c;
```

This is obviously valid code in C++, since the different variables of the addition are all fundamental types. Nevertheless, it is not so obvious that we could perform an operation similar to the following one:

```
struct {
  string product;
  float price;
} a, b, c;
a = b + c;
```

In fact, this will cause a compilation error, since we have not defined the behavior our class should have with addition operations. However, thanks to the C++ feature to overload operators, we can design classes able to perform operations using standard operators. Here is a list of all the operators that can be overloaded:

Overloadable operators
+ - * / = < > += -= *= /= << >> <<= >>= == != <= >= ++ -- % & ^ ! | ~ &= ^= |= && || %= [] () , ->* -> new delete new[] delete[]

To overload an operator in order to use it with classes we declare operator functions, which are regular functions whose names are the operator keyword followed by the operator sign that we want to overload. The format is:

type operator sign (parameters) { /*...*/ }

Here you have an example that overloads the addition operator (+). We are going to create a class to store bidimensional vectors and then we are going to add two of them: a(3,1) and b(1,2). The addition of two bidimensional vectors is an operation as simple as adding the two x coordinates to obtain the resulting x coordinate and adding the two y coordinates to obtain the resulting y. In this case the result will be (3+1,1+2) = (4,3).

```
// vectors: overloading operators example
#include <iostream>
using namespace std;

class CVector {
  public:
    int x,y;
    CVector () {};
    CVector (int,int);
    CVector operator + (CVector);
};

CVector::CVector (int a, int b) {
  x = a;
  y = b;
}
```

```
CVector CVector::operator+ (CVector param) {
  CVector temp;
  temp.x = x + param.x;
  temp.y = y + param.y;
  return (temp);
}

int main () {
  CVector a (3,1);
  CVector b (1,2);
  CVector c;
  c = a + b;
  cout << c.x << "," << c.y;
  return 0;
}
```

Output would be:

4,3

It may be a little confusing to see so many times the CVector identifier. But, consider that some of them refer to the class name (type) CVector and some others are functions with that name (constructors must have the same name as the class). Do not confuse them:

```
CVector (int, int);          // function name CVector (constructor)
CVector operator+ (CVector);  // function returns a CVector
```

The function operator+ of class CVector is the one that is in charge of overloading the addition operator (+). This function can be called either implicitly using the operator, or explicitly using the function name:

```
c = a + b;
c = a.operator+ (b);
```

Both expressions are equivalent.

Notice also that we have included the empty constructor (without parameters) and we have defined it with an empty block:

```
CVector () { };
```

This is necessary, since we have explicitly declared another constructor:

```
CVector (int, int);
```

And when we explicitly declare any constructor, with any number of parameters, the default constructor with no parameters that the compiler can declare automatically is not declared, so we need to declare it ourselves in order to be able to construct objects of this type without parameters. Otherwise, the declaration:

```
CVector c;
```

included in main() would not have been valid.

One needs to be warned that an empty block is a bad implementation for a constructor, since it does not fulfill the minimum functionality that is generally expected from a constructor, which is the initialization of all the member variables in its class. In our case this constructor leaves the variables x and y undefined. Therefore, a more advisable definition would have been something similar to this:

```
CVector () { x=0; y=0; };
```

which in order to simplify and show only the point of the code we have not included in the example.

As well as a class includes a default constructor and a copy constructor even if they are not declared, it also includes a default definition for the assignation operator (=) with the class itself as parameter. The behavior which is defined by default is to copy the whole content of the data members of the object passed as argument (the one at the right side of the sign) to the one at the left side:

```
CVector d (2,3);
CVector e;
e = d;          // copy assignment operator
```

The copy assignment operator function is the only operator member function implemented by default. Of course, you can redefine it to any other functionality that you want, like for example, copy only certain class members or perform additional initialization procedures.

The overload of operators does not force its operation to bear a relation to the mathematical or usual meaning of the operator, although it is recommended. For example, the code may not be very intuitive if you use operator + to subtract two classes or operator== to fill with zeros a class, although it is perfectly possible to do so.

Although the prototype of a function operator+ can seem obvious since it takes what is at the right side of the operator as the parameter for the operator member function of the object at its left side, other operators may not be so obvious. Here you have a table with a summary on how the different operator functions have to be declared (replace @ by the operator in each case):

Expression	Operator	Member function	Global function
@a	+ - * & ! ~ ++ --	A::operator@()	operator@(A)
a@	++ --	A::operator@(int)	operator@(A,int)
a@b	+ - * / % ^ & | < > == != <= >= << >> && || ,	A::operator@ (B)	operator@(A,B)
a@b	= += -= *= /= %= ^= &= |= <<= >>= []	A::operator@ (B)	-
a(b, c...)	()	A::operator() (B, C...)	-
a->x	->	A::operator->()	-

Where a is an object of class A, b is an object of class B and c is an object of class C.

You can see in this panel that there are two ways to overload some class operators: as a member function and as a global function. Its use is indistinct, nevertheless we remind you that functions that are not members of a class cannot access the private or protected members of that class unless the global function is its friend (friendship is explained later).

The keyword this

The keyword this represents a pointer to the object whose member function is being executed. It is a pointer to the object itself.

One of its uses can to check if a parameter passed to a member function is the object itself. For example,

```
// this
#include <iostream>
using namespace std;

class CDummy {
  public:
    int isitme (CDummy& param);
};

int CDummy::isitme (CDummy& param)
{
  if (&param == this) return true;
  else return false;
}

int main () {
```

```
  CDummy a;
  CDummy* b = &a;
  if ( b->isitme(a) )
    cout << "yes, &a is b";
  return 0;
}
```

```
  Output would be:
yes, &a is b
```

It is also frequently used in operator= member functions that return objects by reference (avoiding the use of temporary objects). Following with the vector's examples seen before we could have written an operator= function similar to this one:

```
CVector& CVector::operator= (const CVector& param)
{
  x=param.x;
  y=param.y;
  return *this;
}
```

In fact this function is very similar to the code that the compiler generates implicitly for this class if we do not include an operator= member function to copy objects of this class.

Static members

A class can contain static members, either data or functions.

Static data members of a class are also known as "class variables", because there is only one unique value for all the objects of that same class. Their content is not different from one object of this class to another.

For example, it may be used for a variable within a class that can contain a counter with the number of objects of that class that have been created, as in the following example:

```
// static members in classes
#include <iostream>
using namespace std;

class CDummy {
  public:
    static int n;
    CDummy () { n++; };
```

```
    ~CDummy () { n--; };
};

int CDummy::n=0;

int main () {
  CDummy a;
  CDummy b[5];
  CDummy * c = new CDummy;
  cout << a.n << endl;
  delete c;
  cout << CDummy::n << endl;
  return 0;
}
```

Output would be:

```
7
6
```

In fact, static members have the same properties as global variables but they enjoy class scope. For that reason, and to avoid them to be declared several times, we can only include the prototype (its declaration) in the class declaration but not its definition (its initialization). In order to initialize a static data-member we must include a formal definition outside the class, in the global scope, as in the previous example:

```
int CDummy::n=0;
```

Because it is a unique variable value for all the objects of the same class, it can be referred to as a member of any object of that class or even directly by the class name (of course this is only valid for static members):

```
cout << a.n;
cout << CDummy::n;
```

These two calls included in the previous example are referring to the same variable: the static variable n within class CDummy shared by all objects of this class.

Once again, we remind you that in fact it is a global variable. The only difference is its name and possible access restrictions outside its class.

Just as we may include static data within a class, we can also include static functions. They represent the same: they are global functions that are called as if they were object members of a given class. They can only refer to static data, in no case to non-static members of the class, as well as they do not allow the use of the keyword this, since it makes reference to an object pointer and these functions in fact are not members of any object but direct members of the class.

Friendship and inheritance

Friend functions

In principle, private and protected members of a class cannot be accessed from outside the same class in which they are declared. However, this rule does not affect friends.

Friends are functions or classes declared as such.

If we want to declare an external function as friend of a class, thus allowing this function to have access to the private and protected members of this class, we do it by declaring a prototype of this external function within the class, and preceding it with the keyword friend:

```
// friend functions
#include <iostream>
using namespace std;

class CRectangle {
    int width, height;
  public:
    void set_values (int, int);
    int area () {return (width * height);}
    friend CRectangle duplicate (CRectangle);
};

void CRectangle::set_values (int a, int b) {
  width = a;
  height = b;
}

CRectangle duplicate (CRectangle rectparam)
{
  CRectangle rectres;
  rectres.width = rectparam.width*2;
  rectres.height = rectparam.height*2;
  return (rectres);
}

int main () {
  CRectangle rect, rectb;
  rect.set_values (2,3);
  rectb = duplicate (rect);
```

```
  cout << rectb.area();
  return 0;
}
```

```
  Output would be:
24
```

The duplicate function is a friend of CRectangle. From within that function we have been able to access the members width and height of different objects of type CRectangle, which are private members. Notice that neither in the declaration of duplicate() nor in its later use in main() have we considered duplicate a member of class CRectangle. It isn't! It simply has access to its private and protected members without being a member.

The friend functions can serve, for example, to conduct operations between two different classes. Generally, the use of friend functions is out of an object-oriented programming methodology, so whenever possible it is better to use members of the same class to perform operations with them. Such as in the previous example, it would have been shorter to integrate duplicate() within the class CRectangle.

Friend classes

Just as we have the possibility to define a friend function, we can also define a class as friend of another one, granting that second class access to the protected and private members of the first one.

```
// friend class
#include <iostream>
using namespace std;

class CSquare;

class CRectangle {
    int width, height;
  public:
    int area ()
      {return (width * height);}
    void convert (CSquare a);
};

class CSquare {
  private:
    int side;
  public:
    void set_side (int a)
      {side=a;}
    friend class CRectangle;
};
```

```
void CRectangle::convert (CSquare a) {
  width = a.side;
  height = a.side;
}

int main () {
  CSquare sqr;
  CRectangle rect;
  sqr.set_side(4);
  rect.convert(sqr);
  cout << rect.area();
  return 0;
}
```

Output would be:
16

In this example, we have declared CRectangle as a friend of CSquare so that CRectangle member functions could have access to the protected and private members of CSquare, more concretely to CSquare::side, which describes the side width of the square.

You may also see something new at the beginning of the program: an empty declaration of class CSquare. This is necessary because within the declaration of CRectangle we refer to CSquare (as a parameter in convert()). The definition of CSquare is included later, so if we did not include a previous empty declaration for CSquare this class would not be visible from within the definition of CRectangle.

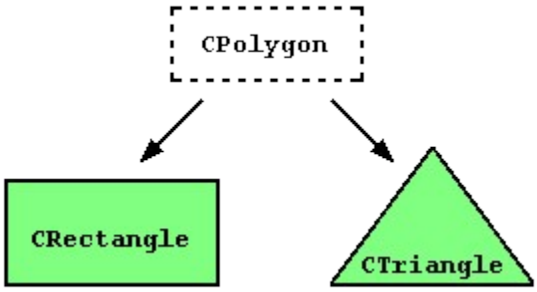
Consider that friendships are not corresponded if we do not explicitly specify so. In our example, CRectangle is considered as a friend class by CSquare, but CRectangle does not consider CSquare to be a friend, so CRectangle can access the protected and private members of CSquare but not the reverse way. Of course, we could have declared also CSquare as friend of CRectangle if we wanted to.

Another property of friendships is that they are not transitive: The friend of a friend is not considered to be a friend unless explicitly specified.

Inheritance between classes

A key feature of C++ classes is inheritance. Inheritance allows to create classes which are derived from other classes, so that they automatically include some of its "parent's" members, plus its own. For example, we are going to suppose that we want to declare a series of classes that describe polygons like our CRectangle, or like CTriangle. They have certain common properties, such as both can be described by means of only two sides: height and base.

This could be represented in the world of classes with a class CPolygon from which we would derive the two other ones: CRectangle and CTriangle.



The class CPolygon would contain members that are common for both types of polygon. In our case: width and height. And CRectangle and CTriangle would be its derived classes, with specific features that are different from one type of polygon to the other.

Classes that are derived from others inherit all the accessible members of the base class. That means that if a base class includes a member A and we derive it to another class with another member called B, the derived class will contain both members A and B.

In order to derive a class from another, we use a colon (:) in the declaration of the derived class using the following format:

```
class derived_class_name: public base_class_name
{ /*...*/ };
```

Where derived_class_name is the name of the derived class and base_class_name is the name of the class on which it is based. The public access specifier may be replaced by any one of the other access specifiers protected and private. This access specifier describes the minimum access level for the members that are inherited from the base class.

```
// derived classes
#include <iostream>
using namespace std;

class CPolygon {
  protected:
    int width, height;
  public:
    void set_values (int a, int b)
      { width=a; height=b;}
  };

class CRectangle: public CPolygon {
```

```
  public:
    int area ()
      { return (width * height); }
  };

class CTriangle: public CPolygon {
  public:
    int area ()
      { return (width * height / 2); }
  };

int main () {
  CRectangle rect;
  CTriangle trgl;
  rect.set_values (4,5);
  trgl.set_values (4,5);
  cout << rect.area() << endl;
  cout << trgl.area() << endl;
  return 0;
}
```

Output would be:

```
20
10
```

The objects of the classes CRectangle and CTriangle each contain members inherited from CPolygon. These are: width, height and set_values().

The protected access specifier is similar to private. Its only difference occurs in fact with inheritance. When a class inherits from another one, the members of the derived class can access the protected members inherited from the base class, but not its private members.

Since we wanted width and height to be accessible from members of the derived classes CRectangle and CTriangle and not only by members of CPolygon, we have used protected access instead of private.

We can summarize the different access types according to who can access them in the following way:

Access	public	protected	private
members of the same class	yes	yes	yes
members of derived classes	yes	yes	no
Not members	yes	no	no

Where "not members" represent any access from outside the class, such as from main(), from another class or from a function.

In our example, the members inherited by CRectangle and CTriangle have the same access permissions as they had in their base class CPolygon:

```
CPolygon::width          // protected access
CRectangle::width        // protected access

CPolygon::set_values()   // public access
CRectangle::set_values() // public access
```

This is because we have used the public keyword to define the inheritance relationship on each of the derived classes:

```
class CRectangle: public CPolygon { ... }
```

This public keyword after the colon (:) denotes the minimum access level for all the members inherited from the class that follows it (in this case CPolygon). Since public is the most accessible level, by specifying this keyword the derived class will inherit all the members with the same levels they had in the base class.

If we specify a more restrictive access level like protected, all public members of the base class are inherited as protected in the derived class. Whereas if we specify the most restricting of all access levels: private, all the base class members are inherited as private.

For example, if daughter was a class derived from mother that we defined as:

```
class daughter: protected mother;
```

This would set protected as the maximum access level for the members of daughter that it inherited from mother. That is, all members that were public in mother would become protected in daughter. Of course, this would not restrict daughter to declare its own public members. That maximum access level is only set for the members inherited from mother.

If we do not explicitly specify any access level for the inheritance, the compiler assumes private for classes declared with class keyword and public for those declared with struct.

What is inherited from the base class?

In principle, a derived class inherits every member of a base class except:

its constructor and its destructor

its operator=() members

its friends

Although the constructors and destructors of the base class are not inherited themselves, its default constructor (i.e., its constructor with no parameters) and its destructor are always called when a new object of a derived class is created or destroyed.

If the base class has no default constructor or you want that an overloaded constructor is called when a new derived object is created, you can specify it in each constructor definition of the derived class:

derived_constructor_name (parameters) : base_constructor_name (parameters) {...}

For example:

```
// constructors and derived classes
#include <iostream>
using namespace std;

class mother {
  public:
    mother ()
      { cout << "mother: no parameters\n"; }
    mother (int a)
      { cout << "mother: int parameter\n"; }
};

class daughter : public mother {
  public:
    daughter (int a)
      { cout << "daughter: int parameter\n\n"; }
};

class son : public mother {
  public:
    son (int a) : mother (a)
      { cout << "son: int parameter\n\n"; }
};

int main () {
  daughter cynthia (0);
  son daniel(0);

  return 0;
```

```
}
```

```
Output would be:
mother: no parameters
daughter: int parameter

mother: int parameter
son: int parameter
```

Notice the difference between which mother's constructor is called when a new daughter object is created and which when it is a son object. The difference is because the constructor declaration of daughter and son:

```
daughter (int a)          // nothing specified: call default
son (int a) : mother (a)  // constructor specified: call this
```

Multiple inheritance

In C++ it is perfectly possible that a class inherits members from more than one class. This is done by simply separating the different base classes with commas in the derived class declaration. For example, if we had a specific class to print on screen (COutput) and we wanted our classes CRectangle and CTriangle to also inherit its members in addition to those of CPolygon we could write:

```
class CRectangle: public CPolygon, public COutput;
class CTriangle: public CPolygon, public COutput;
```

here is the complete example:

```
// multiple inheritance
#include <iostream>
using namespace std;

class CPolygon {
  protected:
    int width, height;
  public:
    void set_values (int a, int b)
      { width=a; height=b;}
  };

class COutput {
  public:
    void output (int i);
  };
```

```
void COutput::output (int i) {
  cout << i << endl;
  }

class CRectangle: public CPolygon, public COutput {
  public:
    int area ()
      { return (width * height); }
  };

class CTriangle: public CPolygon, public COutput {
  public:
    int area ()
      { return (width * height / 2); }
  };

int main () {
  CRectangle rect;
  CTriangle trgl;
  rect.set_values (4,5);
  trgl.set_values (4,5);
  rect.output (rect.area());
  trgl.output (trgl.area());
  return 0;
}
```

Output would be:

```
20
10
```

Polymorphism

Before getting into this section, it is recommended that you have a proper understanding of pointers and class inheritance. If any of the following statements seem strange to you, you should review the indicated sections:

Statement:	Explained in:
int a::b(c) {};	Classes
a->b	Pointers
Class a: public b;	Friendship and inheritance

Pointers to base class

One of the key features of derived classes is that a pointer to a derived class is type-compatible with a pointer to its base class. Polymorphism is the art of taking advantage of this simple but powerful and versatile feature, that brings Object Oriented Methodologies to its full potential.

We are going to start by rewriting our program about the rectangle and the triangle of the previous section taking into consideration this pointer compatibility property:

```
// pointers to base class
#include <iostream>
using namespace std;

class CPolygon {
  protected:
    int width, height;
  public:
    void set_values (int a, int b)
      { width=a; height=b; }
  };

class CRectangle: public CPolygon {
  public:
    int area ()
      { return (width * height); }
  };

class CTriangle: public CPolygon {
  public:
    int area ()
      { return (width * height / 2); }
  };

int main () {
  CRectangle rect;
  CTriangle trgl;
  CPolygon * ppoly1 = &rect;
  CPolygon * ppoly2 = &trgl;
  ppoly1->set_values (4,5);
  ppoly2->set_values (4,5);
  cout << rect.area() << endl;
  cout << trgl.area() << endl;
  return 0;
}
```

Output would be:

```
20
10
```

In function main, we create two pointers that point to objects of class CPolygon (ppoly1 and ppoly2). Then we assign references to rect and trgl to these pointers, and because both are objects of classes derived from CPolygon, both are valid assignations.

The only limitation in using *ppoly1 and *ppoly2 instead of rect and trgl is that both *ppoly1 and *ppoly2 are of type CPolygon* and therefore we can only use these pointers to refer to the members that CRectangle and CTriangle inherit from CPolygon. For that reason when we call the area() members at the end of the program we have had to use directly the objects rect and trgl instead of the pointers *ppoly1 and *ppoly2.

In order to use area() with the pointers to class CPolygon, this member should also have been declared in the class CPolygon, and not only in its derived classes, but the problem is that CRectangle and CTriangle implement different versions of area, therefore we cannot implement it in the base class. This is when virtual members become handy:

Virtual members

A member of a class that can be redefined in its derived classes is known as a virtual member. In order to declare a member of a class as virtual, we must precede its declaration with the keyword virtual:

```
// virtual members
#include <iostream>
using namespace std;

class CPolygon {
  protected:
    int width, height;
  public:
    void set_values (int a, int b)
      { width=a; height=b; }
    virtual int area ()
      { return (0); }
  };

class CRectangle: public CPolygon {
  public:
    int area ()
      { return (width * height); }
```

```
  };

class CTriangle: public CPolygon {
  public:
    int area ()
      { return (width * height / 2); }
  };

int main () {
  CRectangle rect;
  CTriangle trgl;
  CPolygon poly;
  CPolygon * ppoly1 = &rect;
  CPolygon * ppoly2 = &trgl;
  CPolygon * ppoly3 = &poly;
  ppoly1->set_values (4,5);
  ppoly2->set_values (4,5);
  ppoly3->set_values (4,5);
  cout << ppoly1->area() << endl;
  cout << ppoly2->area() << endl;
  cout << ppoly3->area() << endl;
  return 0;
}
```

Output would be:

```
20
10
0
```

Now the three classes (CPolygon, CRectangle and CTriangle) have all the same members: width, height, set_values() and area().

The member function area() has been declared as virtual in the base class because it is later redefined in each derived class. You can verify if you want that if you remove this virtual keyword from the declaration of area() within CPolygon, and then you run the program the result will be 0 for the three polygons instead of 20, 10 and 0. That is because instead of calling the corresponding area() function for each object (CRectangle::area(), CTriangle::area() and CPolygon::area(), respectively), CPolygon::area() will be called in all cases since the calls are via a pointer whose type is CPolygon*.

Therefore, what the virtual keyword does is to allow a member of a derived class with the same name as one in the base class to be appropriately called from a pointer, and more precisely when the type of the pointer is a pointer to the base class but is pointing to an object of the derived class, as in the above example.

A class that declares or inherits a virtual function is called a polymorphic class.

Note that despite of its virtuality, we have also been able to declare an object of type CPolygon and to call its own area() function, which always returns 0.

Abstract base classes

Abstract base classes are something very similar to our CPolygon class of our previous example. The only difference is that in our previous example we have defined a valid area() function with a minimal functionality for objects that were of class CPolygon (like the object poly), whereas in an abstract base classes we could leave that area() member function without implementation at all. This is done by appending =0 (equal to zero) to the function declaration.

An abstract base CPolygon class could look like this:

```
// abstract class CPolygon
class CPolygon {
  protected:
    int width, height;
  public:
    void set_values (int a, int b)
      { width=a; height=b; }
    virtual int area () =0;
};
```

Notice how we appended =0 to virtual int area () instead of specifying an implementation for the function. This type of function is called a pure virtual function, and all classes that contain at least one pure virtual function are abstract base classes.

The main difference between an abstract base class and a regular polymorphic class is that because in abstract base classes at least one of its members lacks implementation we cannot create instances (objects) of it.

But a class that cannot instantiate objects is not totally useless; We can create pointers to it and take advantage of all its polymorphic abilities. Therefore a declaration like:

```
CPolygon poly;
```

would not be valid for the abstract base class we have just declared, because tries to instantiate an object. Nevertheless, the following pointers:

```
CPolygon * ppoly1;
CPolygon * ppoly2;
```

would be perfectly valid.

This is so for as long as CPolygon includes a pure virtual function and therefore it's an abstract base class. However, pointers to this abstract base class can be used to point to objects of derived classes.

Here you have the complete example:

```
// abstract base class
#include <iostream>
using namespace std;

class CPolygon {
  protected:
    int width, height;
  public:
    void set_values (int a, int b)
      { width=a; height=b; }
    virtual int area (void) =0;
  };

class CRectangle: public CPolygon {
  public:
    int area (void)
      { return (width * height); }
  };

class CTriangle: public CPolygon {
  public:
    int area (void)
      { return (width * height / 2); }
  };

int main () {
  CRectangle rect;
  CTriangle trgl;
  CPolygon * ppoly1 = &rect;
  CPolygon * ppoly2 = &trgl;
  ppoly1->set_values (4,5);
  ppoly2->set_values (4,5);
  cout << ppoly1->area() << endl;
  cout << ppoly2->area() << endl;
  return 0;
}
```

Output would be:

```
20
10
```

If you review the program you will notice that we refer to objects of different but related classes using a unique type of pointer (CPolygon*). This can be tremendously useful. For example, now we can create a function member of the abstract base class CPolygon that is able to print on screen the result of the area() function even though CPolygon itself has no implementation for this function:

```
// pure virtual members can be called
// from the abstract base class
#include <iostream>
using namespace std;

class CPolygon {
  protected:
    int width, height;
  public:
    void set_values (int a, int b)
      { width=a; height=b; }
    virtual int area (void) =0;
    void printarea (void)
      { cout << this->area() << endl; }
  };

class CRectangle: public CPolygon {
  public:
    int area (void)
      { return (width * height); }
  };

class CTriangle: public CPolygon {
  public:
    int area (void)
      { return (width * height / 2); }
  };

int main () {
  CRectangle rect;
  CTriangle trgl;
  CPolygon * ppoly1 = &rect;
  CPolygon * ppoly2 = &trgl;
  ppoly1->set_values (4,5);
  ppoly2->set_values (4,5);
  ppoly1->printarea();
  ppoly2->printarea();
  return 0;
}
```

Output would be:

```
20
```

10

Virtual members and abstract classes grant C++ the polymorphic characteristics that make object-oriented programming such a useful instrument in big projects. Of course, we have seen very simple uses of these features, but these features can be applied to arrays of objects or dynamically allocated objects.

Let's end with the same example again, but this time with objects that are dynamically allocated:

```
// dynamic allocation and polymorphism
#include <iostream>
using namespace std;

class CPolygon {
  protected:
    int width, height;
  public:
    void set_values (int a, int b)
      { width=a; height=b; }
    virtual int area (void) =0;
    void printarea (void)
      { cout << this->area() << endl; }
  };

class CRectangle: public CPolygon {
  public:
    int area (void)
      { return (width * height); }
  };

class CTriangle: public CPolygon {
  public:
    int area (void)
      { return (width * height / 2); }
  };

int main () {
  CPolygon * ppoly1 = new CRectangle;
  CPolygon * ppoly2 = new CTriangle;
  ppoly1->set_values (4,5);
  ppoly2->set_values (4,5);
  ppoly1->printarea();
  ppoly2->printarea();
  delete ppoly1;
  delete ppoly2;
  return 0;
}
```

Output would be:
20
10

Notice that the ppoly pointers:

```
CPolygon * ppoly1 = new CRectangle;
CPolygon * ppoly2 = new CTriangle;
```

are declared being of type pointer to CPolygon but the objects dynamically allocated have been declared having the derived class type directly.

Type Casting

Converting an expression of a given type into another type is known as type-casting. We have already seen some ways to type cast:

Implicit conversion

Implicit conversions do not require any operator. They are automatically performed when a value is copied to a compatible type. For example:

```
short a=2000;
int b;
b=a;
```

Here, the value of a has been promoted from short to int and we have not had to specify any type-casting operator. This is known as a standard conversion. Standard conversions affect fundamental data types, and allow conversions such as the conversions between numerical types (short to int, int to float, double to int...), to or from bool, and some pointer conversions. Some of these conversions may imply a loss of precision, which the compiler can signal with a warning. This can be avoided with an explicit conversion.

Implicit conversions also include constructor or operator conversions, which affect classes that include specific constructors or operator functions to perform conversions. For example:

```
class A {};
class B { public: B (A a) {} };

A a;
B b=a;
```

Here, a implicit conversion happened between objects of class A and class B, because B has a constructor that takes an object of class A as parameter. Therefore implicit conversions from A to B are allowed.

Explicit conversion

C++ is a strong-typed language. Many conversions, specially those that imply a different interpretation of the value, require an explicit conversion. We have already seen two notations for explicit type conversion: functional and c-like casting:

```
short a=2000;
int b;
b = (int) a;    // c-like cast notation
b = int (a);    // functional notation
```

The functionality of these explicit conversion operators is enough for most needs with fundamental data types. However, these operators can be applied indiscriminately on classes and pointers to classes, which can lead to code that while being syntactically correct can cause runtime errors. For example, the following code is syntactically correct:

```
// class type-casting
#include <iostream>
using namespace std;

class CDummy {
   float i,j;
};

class CAddition {
        int x,y;
  public:
        CAddition (int a, int b) { x=a; y=b; }
        int result() { return x+y;}
};

int main () {
  CDummy d;
  CAddition * padd;
  padd = (CAddition*) &d;
  cout << padd->result();
  return 0;
}
```

The program declares a pointer to CAddition, but then it assigns to it a reference to an object of another incompatible type using explicit type-casting:

```
padd = (CAddition*) &d;
```

Traditional explicit type-casting allows to convert any pointer into any other pointer type, independently of the types they point to. The subsequent call to member result will produce either a run-time error or a unexpected result.

In order to control these types of conversions between classes, we have four specific casting operators: dynamic_cast, reinterpret_cast, static_cast and const_cast. Their format is to follow the new type enclosed between angle-brackets (<>) and immediately after, the expression to be converted between parentheses.

dynamic_cast <new_type> (expression)
reinterpret_cast <new_type> (expression)
static_cast <new_type> (expression)
const_cast <new_type> (expression)

The traditional type-casting equivalents to these expressions would be:

(new_type) expression
new_type (expression)

but each one with its own special characteristics:

Preprocessor directives

Preprocessor directives are lines included in the code of our programs that are not program statements but directives for the preprocessor. These lines are always preceded by a pound sign (#). The preprocessor is executed before the actual compilation of code begins, therefore the preprocessor digests all these directives before any code is generated by the statements.

These preprocessor directives extend only across a single line of code. As soon as a newline character is found, the preprocessor directive is considered to end. No semicolon (;) is expected at the end of a preprocessor directive. The only way a preprocessor directive can extend through more than one line is by preceding the newline character at the end of the line by a backslash (\).

macro definitions (#define, #undef)

To define preprocessor macros we can use #define. Its format is:

#define identifier replacement

When the preprocessor encounters this directive, it replaces any occurrence of identifier in the rest of the code by replacement. This replacement can be an expression, a statement, a block or simply anything. The preprocessor does not understand C++, it simply replaces any occurrence of identifier by replacement.

```
#define TABLE_SIZE 100
int table1[TABLE_SIZE];
int table2[TABLE_SIZE];
```

After the preprocessor has replaced TABLE_SIZE, the code becomes equivalent to:

```
int table1[100];
int table2[100];
```

This use of #define as constant definer is already known by us from previuos tutorials, but #define can work also with parameters to define function macros:

```
#define getmax(a,b) a>b?a:b
```

This would replace any occurrence of getmax followed by two arguments by the replacement expression, but also replacing each argument by its identifier, exactly as you would expect if it was a function:

```
// function macro
#include <iostream>
using namespace std;

#define getmax(a,b) ((a)>(b)?(a):(b))

int main()
{
  int x=5, y;
  y= getmax(x,2);
  cout << y << endl;
  cout << getmax(7,x) << endl;
  return 0;
}
```

```
Output would be:
5
7
```

Defined macros are not affected by block structure. A macro lasts until it is undefined with the #undef preprocessor directive:

```
#define TABLE_SIZE 100
```

```
int table1[TABLE_SIZE];
#undef TABLE_SIZE
#define TABLE_SIZE 200
int table2[TABLE_SIZE];
```

This would generate the same code as:

```
int table1[100];
int table2[200];
```

Function macro definitions accept two special operators (# and ##) in the replacement sequence:
If the operator # is used before a parameter is used in the replacement sequence, that parameter is replaced by a string literal (as if it were enclosed between double quotes)

```
#define str(x) #x
cout << str(test);
```

This would be translated into:

```
cout << "test";
```

The operator ## concatenates two arguments leaving no blank spaces between them:

```
#define glue(a,b) a ## b
glue(c,out) << "test";
```

This would also be translated into:

```
cout << "test";
```

Because preprocessor replacements happen before any C++ syntax check, macro definitions can be a tricky feature, but be careful: code that relies heavily on complicated macros may result obscure to other programmers, since the syntax they expect is on many occasions different from the regular expressions programmers expect in C++.

Input/Output with files

C++ provides the following classes to perform output and input of characters to/from files:

- **ofstream:** Stream class to write on files
- **ifstream:** Stream class to read from files
- **fstream:** Stream class to both read and write from/to files.

These classes are derived directly or indirectly from the classes istream, and ostream. We have already used objects whose types were these classes: cin is an object of class istream and cout is an object of class ostream. Therfore, we have already been using classes that are related to our file streams. And in fact, we can use our file streams the same way we are already used to use cin and cout, with the only difference that we have to associate these streams with physical files. Let's see an example:

```
// basic file operations
#include <iostream>
#include <fstream>
using namespace std;

int main () {
  ofstream myfile;
  myfile.open ("example.txt");
  myfile << "Writing this to a file.\n";
  myfile.close();
  return 0;
}
```

```
    Output would be:
  [file example.txt]
 Writing this to a file
```

This code creates a file called example.txt and inserts a sentence into it in the same way we are used to do with cout, but using the file stream myfile instead.

But let's go step by step:

Open a file

The first operation generally performed on an object of one of these classes is to associate it to a real file. This procedure is known as to open a file. An open file is represented within a program by a stream object (an instantiation of one of these classes, in the previous example this was myfile) and any input or output operation performed on this stream object will be applied to the physical file associated to it.

In order to open a file with a stream object we use its member function open():

open (filename, mode);

Where filename is a null-terminated character sequence of type const char * (the same type that string literals have) representing the name of the file to be opened, and mode is an optional parameter with a combination of the following flags:

Ios::in	Open for input operations.
Ios::out	Open for output operations.
Ios::binary	Open in binary mode.
Ios::ate	Set the initial position at the end of the file. If this flag is not set to any value, the initial position is the beginning of the file.
Ios::app	All output operations are performed at the end of the file, appending the content to the current content of the file. This flag can only be used in streams open for output-only operations.
Ios::trunc	If the file opened for output operations already existed before, its previous content is deleted and replaced by the new one.

All these flags can be combined using the bitwise operator OR (|). For example, if we want to open the file example.bin in binary mode to add data we could do it by the following call to member function open():

```
ofstream myfile;
myfile.open ("example.bin", ios::out | ios::app | ios::binary);
```

Each one of the open() member functions of the classes ofstream, ifstream and fstream has a default mode that is used if the file is opened without a second argument:

Class	default mode parameter
ofstream	ios::out
ifstream	ios::in
fstream	ios::in	ios::out

For ifstream and ofstream classes, ios::in and ios::out are automatically and respectivelly assumed, even if a mode that does not include them is passed as second argument to the open() member function.

The default value is only applied if the function is called without specifying any value for the mode parameter. If the function is called with any value in that parameter the default mode is overridden, not combined.

File streams opened in binary mode perform input and output operations independently of any format considerations. Non-binary files are known as text files, and some translations may occur due to formatting of some special characters (like newline and carriage return characters).

Since the first task that is performed on a file stream object is generally to open a file, these three classes include a constructor that automatically calls the open() member function and has the exact same parameters as this member. Therefor, we could also have declared the previous myfile object and conducted the same opening operation in our previous example by writing:

```
ofstream myfile ("example.bin", ios::out | ios::app | ios::binary);
```

Combining object construction and stream opening in a single statement. Both forms to open a file are valid and equivalent.

To check if a file stream was successful opening a file, you can do it by calling to member is_open() with no arguments. This member function returns a bool value of true in the case that indeed the stream object is associated with an open file, or false otherwise:

```
if (myfile.is_open()) { /* ok, proceed with output */ }
```

Closing a file

When we are finished with our input and output operations on a file we shall close it so that its resources become available again. In order to do that we have to call the stream's member function close(). This member function takes no parameters, and what it does is to flush the associated buffers and close the file:

```
myfile.close();
```

Once this member function is called, the stream object can be used to open another file, and the file is available again to be opened by other processes.

In case that an object is destructed while still associated with an open file, the destructor automatically calls the member function close().

Text files

Text file streams are those where we do not include the ios::binary flag in their opening mode. These files are designed to store text and thus all values that we input or output from/to them can suffer some formatting transformations, which do not necessarily correspond to their literal binary value.

Data output operations on text files are performed in the same way we operated with cout:

```
// writing on a text file
#include <iostream>
#include <fstream>
using namespace std;

int main () {
  ofstream myfile ("example.txt");
  if (myfile.is_open())
  {
    myfile << "This is a line.\n";
    myfile << "This is another line.\n";
    myfile.close();
  }
  else cout << "Unable to open file";
  return 0;
}
```

Output would be:

```
[file example.txt]
This is a line.
This is another line.
```

Data input from a file can also be performed in the same way that we did with cin:

```
// reading a text file
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main () {
  string line;
  ifstream myfile ("example.txt");
  if (myfile.is_open())
  {
    while (! myfile.eof() )
    {
      getline (myfile,line);
      cout << line << endl;
    }
    myfile.close();
  }

  else cout << "Unable to open file";
```

```
  return 0;
}
```

Output would be:
This is a line.
This is another line.

This last example reads a text file and prints out its content on the screen. Notice how we have used a new member function, called eof() that returns true in the case that the end of the file has been reached. We have created a while loop that finishes when indeed myfile.eof() becomes true (i.e., the end of the file has been reached).

Checking state flags

In addition to eof(), which checks if the end of file has been reached, other member functions exist to check the state of a stream (all of them return a bool value):

bad()

Returns true if a reading or writing operation fails. For example in the case that we try to write to a file that is not open for writing or if the device where we try to write has no space left.

fail()

Returns true in the same cases as bad(), but also in the case that a format error happens, like when an alphabetical character is extracted when we are trying to read an integer number.

eof()

Returns true if a file open for reading has reached the end.

good()

It is the most generic state flag: it returns false in the same cases in which calling any of the previous functions would return true.

In order to reset the state flags checked by any of these member functions we have just seen we can use the member function clear(), which takes no parameters.

get and put stream pointers

All i/o streams objects have, at least, one internal stream pointer:

ifstream, like istream, has a pointer known as the get pointer that points to the element to be read in the next input operation.

ofstream, like ostream, has a pointer known as the put pointer that points to the location where the next element has to be written.

Finally, fstream, inherits both, the get and the put pointers, from iostream (which is itself derived from both istream and ostream).

These internal stream pointers that point to the reading or writing locations within a stream can be manipulated using the following member functions:

tellg() and tellp()

These two member functions have no parameters and return a value of the member type pos_type, which is an integer data type representing the current position of the get stream pointer (in the case of tellg) or the put stream pointer (in the case of tellp).

seekg() and seekp()

These functions allow us to change the position of the get and put stream pointers. Both functions are overloaded with two different prototypes. The first prototype is:

seekg (position);
seekp (position);

Using this prototype the stream pointer is changed to the absolute position position (counting from the beginning of the file). The type for this parameter is the same as the one returned by functions tellg and tellp: the member type pos_type, which is an integer value.

The other prototype for these functions is:

seekg (offset, direction);
seekp (offset, direction);

Using this prototype, the position of the get or put pointer is set to an offset value relative to some specific point determined by the parameter direction. offset is of the member type off_type, which is also an integer type. And direction is of type seekdir, which is an enumerated type (enum) that determines the point from where offset is counted from, and that can take any of the following values:

ios::beg	offset counted from the beginning of the stream
ios::cur	offset counted from the current position of the stream pointer
ios::end	offset counted from the end of the stream

The following example uses the member functions we have just seen to obtain the size of a file:

```
// obtaining file size
#include <iostream>
#include <fstream>
using namespace std;

int main () {
  long begin,end;
  ifstream myfile ("example.txt");
  begin = myfile.tellg();
  myfile.seekg (0, ios::end);
  end = myfile.tellg();
  myfile.close();
  cout << "size is: " << (end-begin) << " bytes.\n";
  return 0;
}
```

Output would be:
size is: 40 bytes

Binary files

In binary files, to input and output data with the extraction and insertion operators (<< and >>) and functions like getline is not efficient, since we do not need to format any data, and data may not use the separation codes used by text files to separate elements (like space, newline, etc...).

File streams include two member functions specifically designed to input and output binary data sequentially: write and read. The first one (write) is a member function of ostream inherited by ofstream. And read is a member function of istream that is inherited by ifstream. Objects of class fstream have both members. Their prototypes are:

write (memory_block, size);
read (memory_block, size);

Where memory_block is of type "pointer to char" (char*), and represents the address of an array of bytes where the read data elements are stored or from where the data elements to be written are taken. The size parameter is an integer value that specifies the number of characters to be read or written from/to the memory block.

```
// reading a complete binary file
#include <iostream>
#include <fstream>
using namespace std;
```

```
ifstream::pos_type size;
char * memblock;

int main () {
  ifstream file ("example.txt", ios::in|ios::binary|ios::ate);
  if (file.is_open())
  {
    size = file.tellg();
    memblock = new char [size];
    file.seekg (0, ios::beg);
    file.read (memblock, size);
    file.close();

    cout << "the complete file content is in memory";

    delete[] memblock;
  }
  else cout << "Unable to open file";
  return 0;
}
```

Output would be:

```
the complete file content is in memory
```

In this example the entire file is read and stored in a memory block. Let's examine how this is done:

First, the file is open with the ios::ate flag, which means that the get pointer will be positioned at the end of the file. This way, when we call to member tellg(), we will directly obtain the size of the file. Notice the type we have used to declare variable size:

```
ifstream::pos_type size;
```

ifstream::pos_type is a specific type used for buffer and file positioning and is the type returned by file.tellg(). This type is defined as an integer type, therefore we can conduct on it the same operations we conduct on any other integer value, and can safely be converted to another integer type large enough to contain the size of the file. For a file with a size under 2GB we could use int:

```
int size;
size = (int) file.tellg();
```

Once we have obtained the size of the file, we request the allocation of a memory block large enough to hold the entire file:

```
memblock = new char[size];
```

Right after that, we proceed to set the get pointer at the beginning of the file (remember that we opened the file with this pointer at the end), then read the entire file, and finally close it:

```
file.seekg (0, ios::beg);
file.read (memblock, size);
file.close();
```

At this point we could operate with the data obtained from the file. Our program simply announces that the content of the file is in memory and then terminates.

Buffers and Synchronization

When we operate with file streams, these are associated to an internal buffer of type streambuf. This buffer is a memory block that acts as an intermediary between the stream and the physical file. For example, with an ofstream, each time the member function put (which writes a single character) is called, the character is not written directly to the physical file with which the stream is associated. Instead of that, the character is inserted in that stream's intermediate buffer.

When the buffer is flushed, all the data contained in it is written to the physical medium (if it is an output stream) or simply freed (if it is an input stream). This process is called synchronization and takes place under any of the following circumstances:

- **When the file is closed:** before closing a file all buffers that have not yet been flushed are synchronized and all pending data is written or read to the physical medium.
- **When the buffer is full:** Buffers have a certain size. When the buffer is full it is automatically synchronized.
- **Explicitly, with manipulators:** When certain manipulators are used on streams, an explicit synchronization takes place. These manipulators are: flush and endl.
- **Explicitly, with member function sync():** Calling stream's member function sync(), which takes no parameters, causes an immediate synchronization. This function returns an int value equal to -1 if the stream has no associated buffer or in case of failure. Otherwise (if the stream buffer was successfully synchronized) it returns 0.

Introduction to C++ Templates
Class templates are an advanced topic that we will study in more detail in the lessons in Topics in C++. Class templates provide a way to parameterize the types

within a class. This means that rather than explicitly specifying the data type associated with a class member or method, a placeholder is used. When we instantiate an instance of the class, the actual data type is substituted for the parameter, or placeholder. The process of forming a class with specific data types from a class template is called template instantiation.

In addition to parameterizing types, it is possible to use non-type parameters in a class template. These non-type parameters will serve as constants within a particular instance of the class.

Although this may seem complex right now, the basic use of class templates is easy once you see a few examples. In fact, you may be using class templates already in your programs. Many of the container classes, such as vector, in the standard C++ library are implemented as class templates.

The much celebrated CrudeStack class from the last lesson will be used in our examples. For reference, it is reproduced here with the int data type being stored. Typically, when building class templates, it is easier to first write and debug a concrete class. This class is then turned into a template. Here is the concrete "int" version of CrudeStack. we have dropped the "Crude" part of the class name, removed exception handling and slightly modified the push and pop methods in order to simplify the presentation.

```
class Stack {
public:
   Stack() : index(-1) {}
   ~Stack() {}
   void push(int val)
   {
      // Increment index, then store
      data[++index] = val;
   }
   int pop()
   {
      // Retrieve, then decrement index
      return data[index--];
   }

private:
   int data[100];
   int index;
};
```

Class Templates

The Stack class listed on the previous page can store only ints. If we need to store data of other types, such as boolean or double, one approach would be to create new copies of the class for each data type. We could have an IntStack, DoubleStack, FloatStack, and so on. The problem with this approach is that there can be considerable work in creating and maintaining multiple copies of a class that differ only in data type. Class templates provide an alternative. Only a single class

template is created and maintained. The data type is simply a parameter. Here is a class template for the Stack class, along with a short program demonstrating its use. Analysis will follow the code.

```
#include <iostream>
using namespace std;

template <class T>
class Stack {
public:
   Stack() : index(-1) {}
   ~Stack() {}
   void push(T val)
   {
      // Increment index, then store
      data[++index] = val;
   }
   T pop()
   {
      // Retrieve, then decrement index
      return data[index--];
   }

private:
   T data[100];
   int index;
};

int main()
{
   // Template instantiation
   Stack<int> stack;
   int num;
   int den;

   cout << "Enter num: ";
   cin >> num;
   cout << "Enter den: ";
   cin >> den;

   stack.push(num);
   stack.push(den);

   val = stack.pop();
   cout << "popped " << val << endl;
   val = stack.pop();
   cout << "popped " << val << endl;
```

```
    return 0;

}
```

Notice the syntax used for the class template. The keyword template is used, followed by a list of parameters each preceded by the keyword class (or equivalently by the keyword typename). The parameter list is enclosed using less than and greater than symbols. Everywhere the keyword int was used to specify a data type in the Stack class, the parameter T is substituted. This is highlighted in blue.

When the class is instantiated, the data type int is specified within less than and greater than symbols following the class name. "int" will be substituted for "T" as the class is built by the compiler. Creating a stack of another data type is simple. Here are some examples.

```
Stack<float> fstack;
Stack<double> dstack;
```

and so forth.

Note that the parameter can be replaced by built-in data types, library classes (such as string) or user defined classes, provided the class itself can support the data type. The Stack class was designed to handle only built-in data types.

Multiple types may be parameterized within a single template. Here is the syntax for a class template with multiple parameters.

```
template <class Parameter1, class Paramter2, ... >
class ClassName
{
   ....
}

or equivalently

template <typename Parameter1, typename Paramter2, ... >
class ClassName
{
   ....
}
```

The example on the previous page had methods defined within the class definition. Suppose, as if often the case, that methods are declared within the class definition, but defined outside. How do we handle this with class templates? Normally, methods defined outside a class definition are prefixed by the class name so that the compiler may identify their class. With class templates, methods defined outside a class definition are prefixed by a template declaration and by the class name with its parameter list. Here's an example.

```
template <class T>
class Stack {
public:
    Stack() : index(-1) {}
    ~Stack() {}

    // Method declarations
    void push(T val);
    T pop();

private:
    T data[100];
    int index;
};

// Method definitions
template <class T>
void Stack<T>::push(T val)
{
    // Increment index, then store
    data[++index] = val;
}

template <class T>
T Stack<T>::pop()
{
    // Retrieve, then decrement index
    return data[index--];
}
```

Constructors and destructors can also be defined outside of a class definition. Here's how to do this when using class templates.

```
template <class T>
Stack<T>::~Stack()
{
    cout << "Stack Destructor" << endl;
}

template <class T>
Stack<T>::Stack(): index(-1)
{
    cout << "Stack Constructor" << endl;
}
```

Template Non-type Parameters

In addition to parameterizing data types within a class template, it is also possible to parameterize non-type parameters. These non-type parameters will serve as constants within instantiations of the class.

```
#include <iostream>
using namespace std;

template <class T, int size>
class Stack {
public:
   Stack() : index(-1) {}
   ~Stack() {}

   // Method declarations
   void push(T val);
   T pop();

private:
   T data[size];
   int index;
};

// Method definitions
template <class T, int size>
void Stack<T, size>::push(T val)
{
   // Increment index, then store
   data[++index] = val;
}

template <class T, int size>
T Stack<T, size>::pop()
{
   // Retrieve, then decrement index
   return data[index--];
}

int main()
{
   Stack<int, 100> stack;
   int num;
   int den;
   int val;

   cout << "Enter num: ";
   cin >> num;
   cout << "Enter den: ";
```

```
    cin >> den;

    stack.push(num);
    stack.push(den);

    val = stack.pop();
    cout << "popped " << val << endl;
    val = stack.pop();
    cout << "popped " << val << endl;

    return 0;

}
```

Notice that now the stack size is a parameter. We can easily create stacks of different data types and sizes.

```
Stack<int, 500> stack;
Stack<float, 10> stack;
Stack<float, 200> stack;
```

That's all folks!